

Servlet 6.0 및 JSP 3.1 (Tomcat 10 기반)

inky4832@daum.net

1장. 웹 어플리케이션 개발환경

1. JDK 21 설치
2. eclipse 설치
3. Tomcat 설치 및 Eclipse 연동
4. 기본 코드 구성 및 서블릿 생성

1. JDK 21 설치

Servlet6.0 및 JSP3.1 Tomcat10

가. 프로그램 다운로드 (회원가입 및 로그인 필수)

<https://www.oracle.com/java/technologies/>



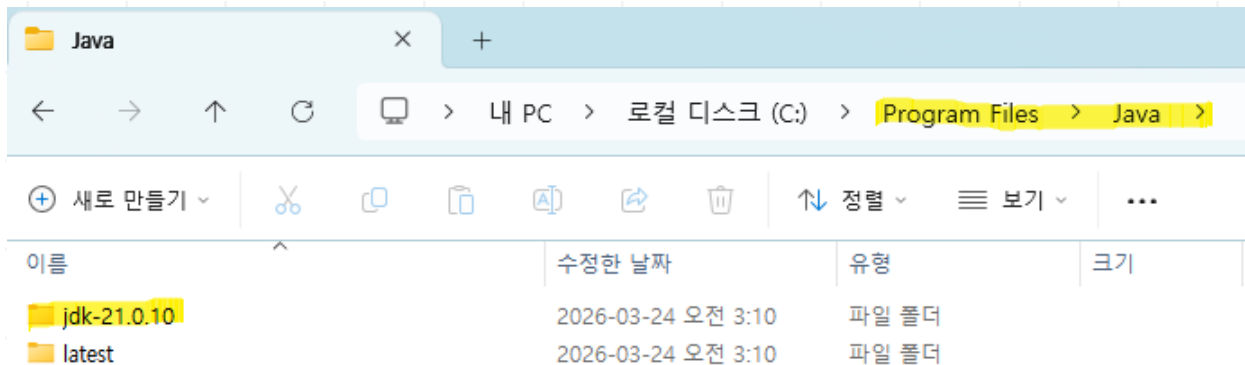
버전은 업데이트 상황에 따라서 변경될 수 있음.

Linux	macOS	Windows
Product/file description		File size
		Download
x64 Compressed Archive		186.54 MB
		https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.zip (sha256)
x64 Installer		165.68 MB
		https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.exe (sha256)
x64 MSI Installer		164.44 MB
		https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.msi (sha256)

1. JDK 21 설치

Servlet6.0 및 JSP3.1 Tomcat10

나. 설치 시작 (기본 설치 권장)



기본적으로 c:\Program Files\Java 폴더에 설치된다.

다. 환경 변수 설정

JAVA_HOME=C:\Program Files\Java\jdk21.0.10

PATH=%JAVA_HOME%\bin;%PATH%

```
C:\Users\User>set JAVA_HOME
JAVA_HOME=C:\Program Files\Java\jdk-21.0.10

C:\Users\User>set PATH
Path=C:\Program Files\Java\jdk-21.0.10\bin;C:\Program Files\Java\jdk-21.0.10\bin
```

```
C:\Users\User>java -version
java version "21.0.10" 2026-01-20 LTS
Java(TM) SE Runtime Environment (build 21.0.10-LTS)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.10-LTS)
```

2. eclipse 설치

Servlet6.0 및 JSP3.1 Tomcat10

가. 프로그램 다운로드

<https://www.eclipse.org/downloads/packages/release/2025-03/r>

Eclipse IDE 2025-03 R Packages

Eclipse IDE for Enterprise Java and Web Developers

508 MB 344,262 DOWNLOADS



Tools for developers working with Java and Web applications, including a Java IDE, tools for JavaScript, TypeScript, JavaServer Pages and Faces, Yaml, Markdown, Web Services, JPA and Data Tools, Maven and Gradle, Git, and more.



Windows: AArch64 | x86_64
macOS: AArch64 | x86_64
Linux: AArch64 | riscv64 | x86_64

Click [here](#) to raise an issue with the Eclipse Web Tools Platform.
Maintainers will move opened issues to the right place.
Click [here](#) to raise an issue with the Eclipse Platform.
Click [here](#) to raise an issue with Maven integration for web projects.
Click [here](#) to raise an issue with Eclipse Wild Web Developer (incubating).

Eclipse IDE for Java Developers

345 MB 260,914 DOWNLOADS



The essential tools for any Java developer, including a Java IDE, a Git client, XML Editor, Maven and Gradle integration



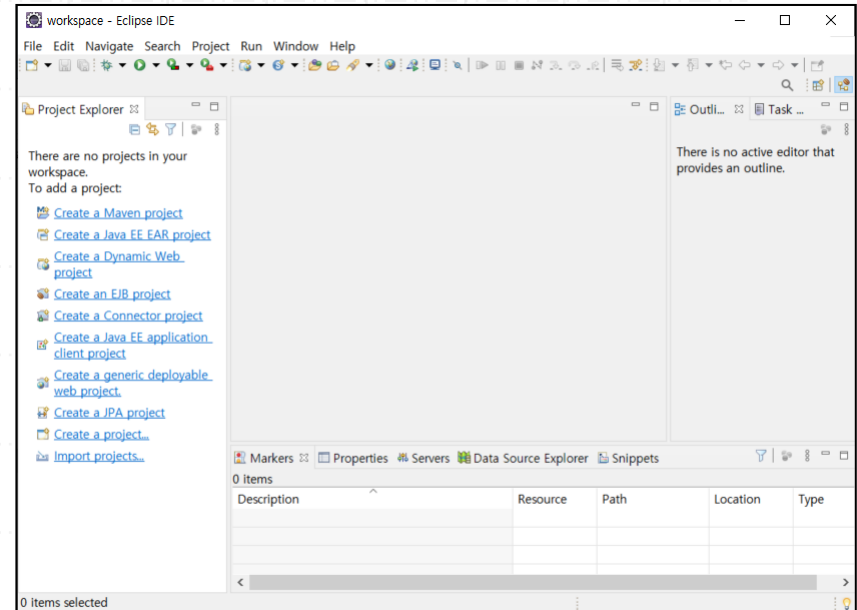
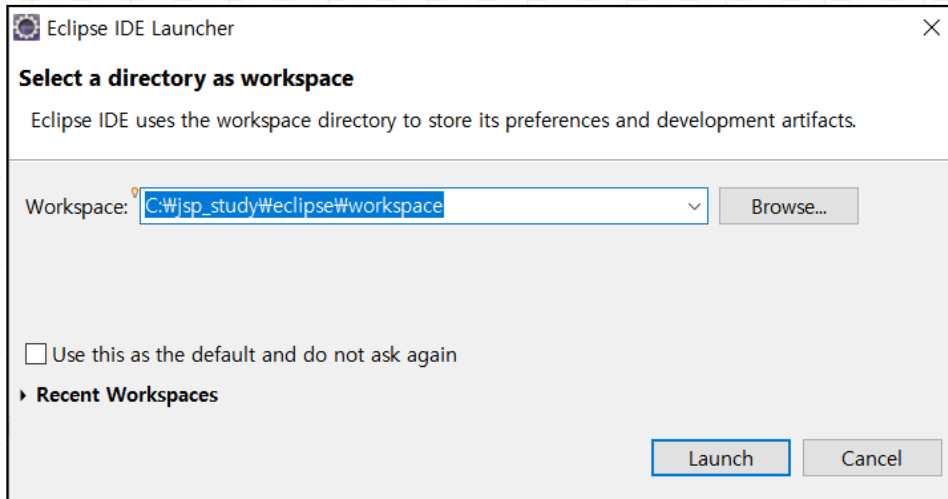
Windows: AArch64 | x86_64
macOS: AArch64 | x86_64
Linux: AArch64 | riscv64 | x86_64

2. eclipse 설치

Servlet6.0 및 JSP3.1 Tomcat10

나. Eclipse 실행 및 workspace 설정

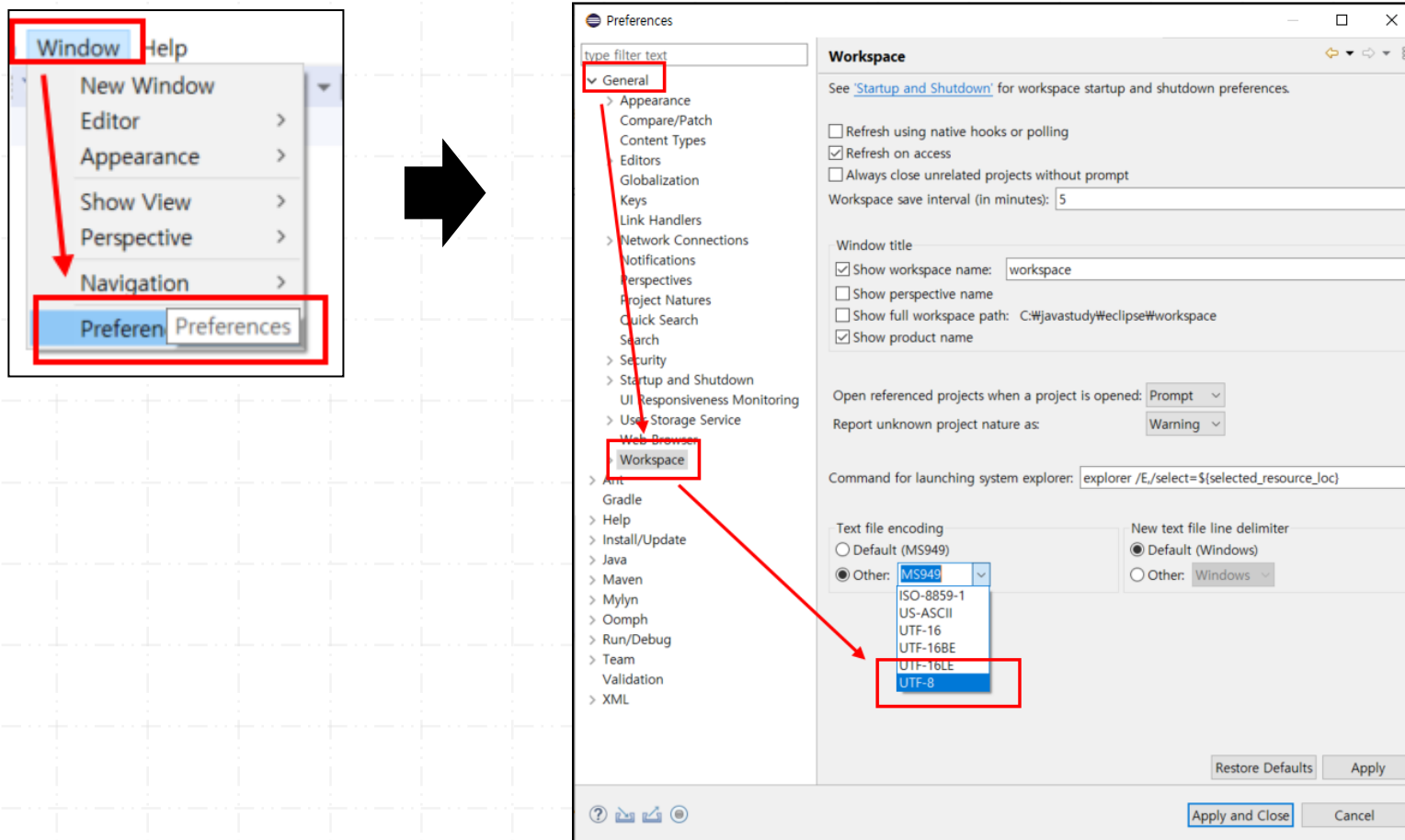
C:\Wjsp_study\Weclipse\workspace 경로를 작업 디렉터리로 설정.



2. eclipse 설치

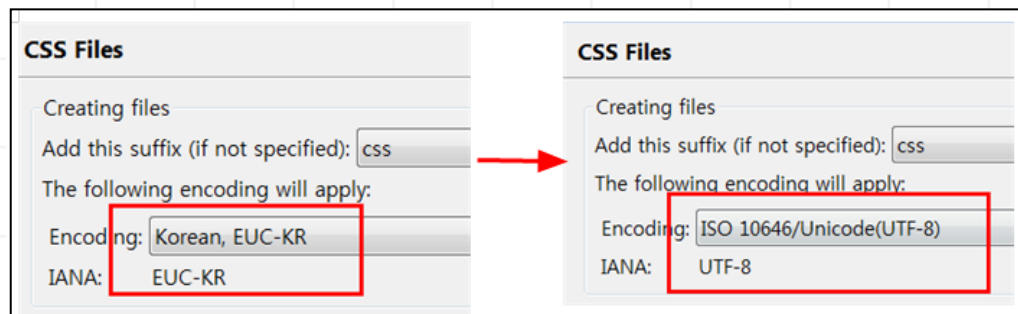
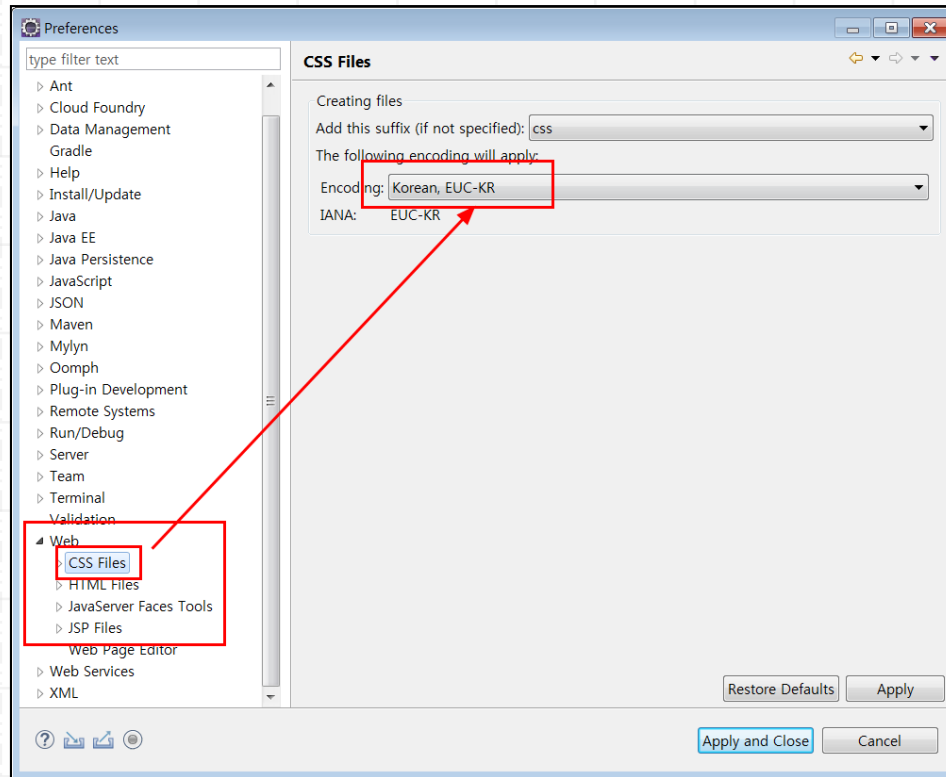
Servlet6.0 및 JSP3.1 Tomcat10

다. utf-8로 인코딩 확인 (구버전인 경우 변경 필요)



2. eclipse 설치

Servlet6.0 및 JSP3.1 Tomcat10

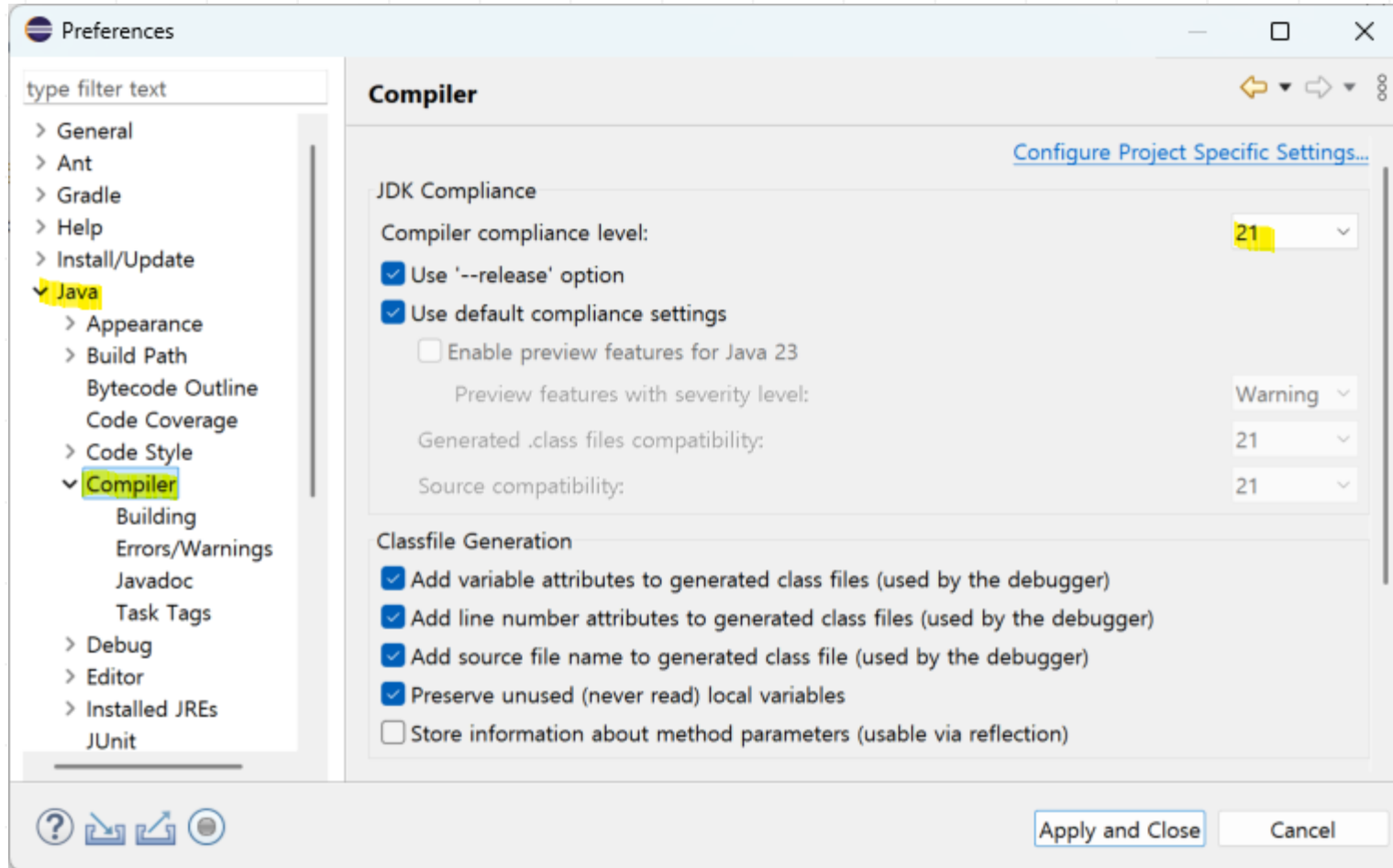


CSS 및 HTML, JSP 모두 UTF-8로 변경

2. eclipse 설치

Servlet6.0 및 JSP3.1 Tomcat10

라. Compiler 확인 (Window > Preferences > Java > Compiler)

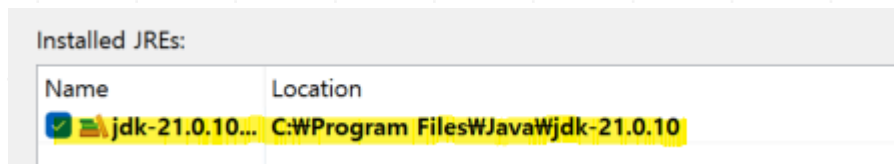
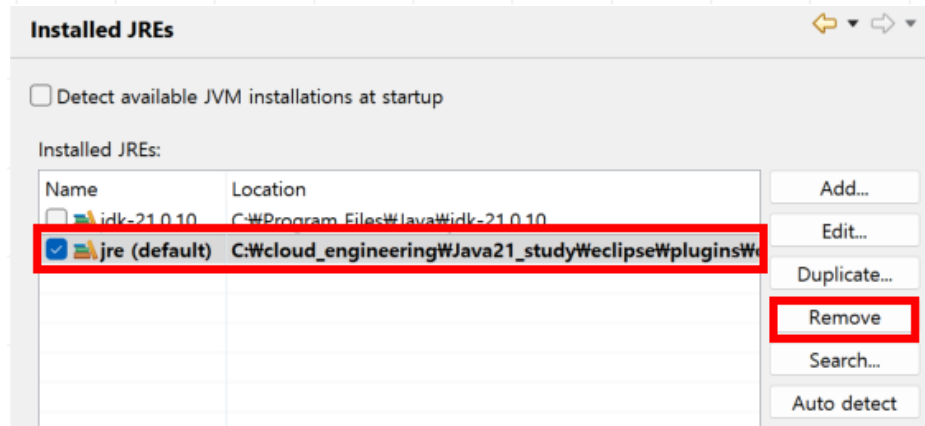


2. eclipse 설치

Servlet6.0 및 JSP3.1 Tomcat10

마. Installed JREs 변경 (Preferences > Java > Installed JREs)

기본으로 설정된 JREs를 Remove 하고 사용자가 설치한 JDK로 설정함.



3. Tomcat 10 설치

Servlet6.0 및 JSP3.1 Tomcat10

가. Tomcat 10 프로그램 다운로드

<https://tomcat.apache.org/>

The screenshot shows the Apache Tomcat download page for version 10.1.28. The page is titled "Apache Tomcat®" and features the Tomcat logo. The main content area is divided into several sections:

- Documentation:** Includes links for Tomcat 11.0 (beta), Tomcat 10.1, Tomcat 9.0, Upgrading, Tomcat Connectors, Tomcat Native 2, Tomcat Native 1.3, Wiki, Migration Guide, Presentations, and Specifications.
- Binary Distributions:** Lists the following options:
 - Core:
 - [zip \(pgp, sha512\)](#)
 - [tar.gz \(pgp, sha512\)](#)
 - [32-bit Windows zip \(pgp, sha512\)](#)
 - [64-bit Windows zip \(pgp, sha512\)](#)
 - [32-bit/64-bit Windows Service Installer \(pgp, sha512\)](#)
 - Full documentation:

The page also includes a search bar, a "COMMUNITY" section for the Hangzhou China 2024 conference, and a "Download" section with links for Tomcat 11 (beta), Tomcat 10, and Tomcat 9. The "Tomcat 10" link is highlighted in yellow.

3. Tomcat 10 설치

Servlet6.0 및 JSP3.1 Tomcat10

서블릿 및 JSP 스펙 버전

<https://tomcat.apache.org/whichversion.html>

Apache Tomcat Versions

Apache Tomcat® is an open source software implementation of a subset of the Jakarta EE (formerly Java EE) technologies. Different versions of Apache Tomcat are available for different versions of the specifications. The mapping between [the specifications](#) and the respective Apache Tomcat versions is:

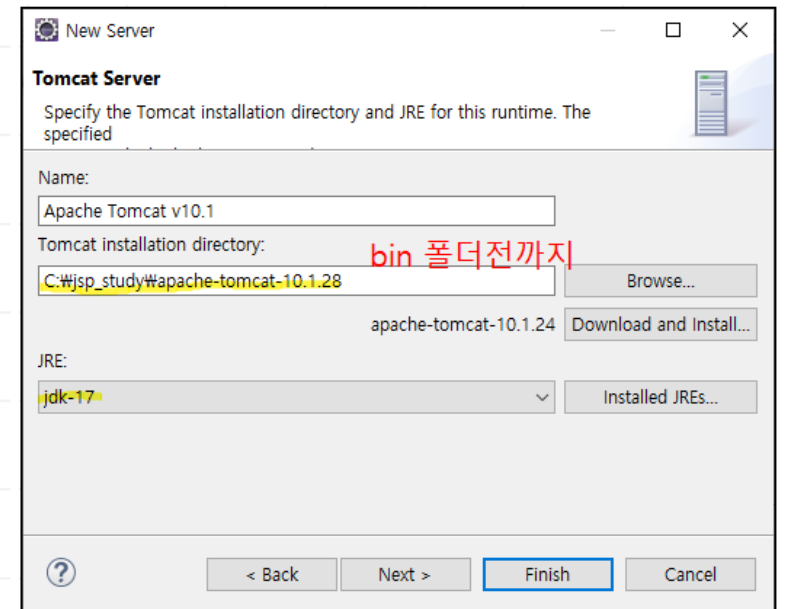
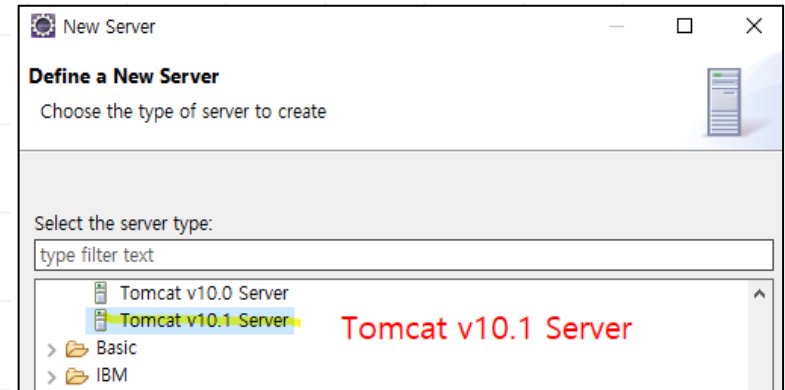
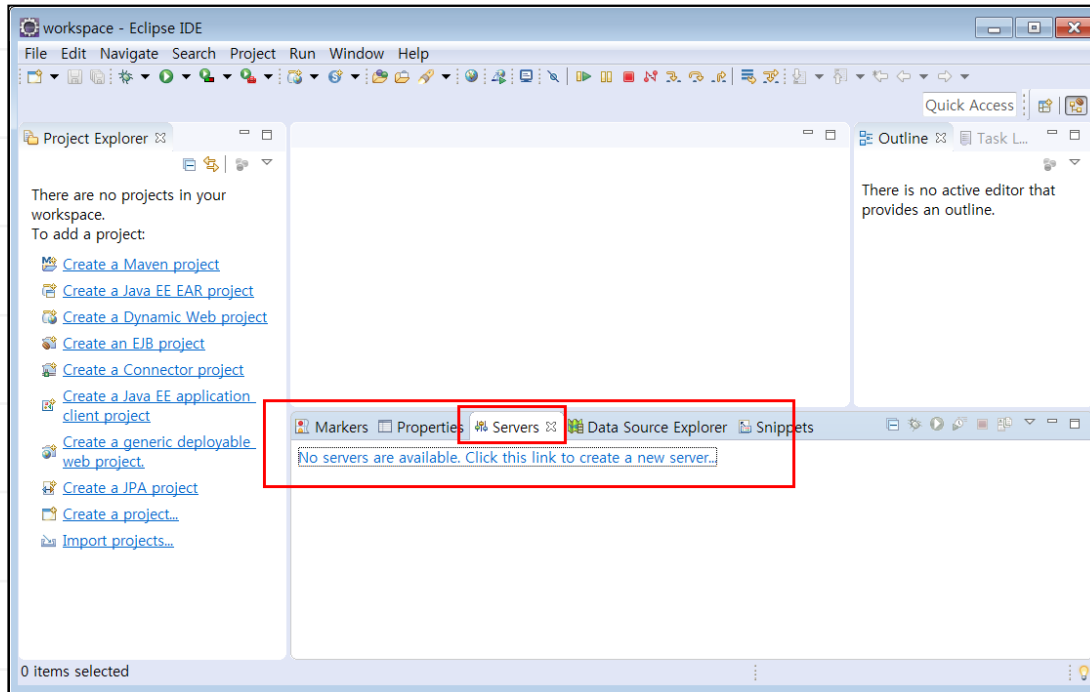
Currently Supported Versions

Servlet Spec	JSP Spec	EL Spec	WebSocket Spec	Authentication Spec (JASPIC)	Apache Tomcat Version	Latest Released Version	Supported Java Versions
6.1	4.0	6.0	2.2	3.1	11.0.x	11.0.0-M24 (beta)	17 and later
6.0	3.1	5.0	2.1	3.0	10.1.x	10.1.28	11 and later
4.0	2.3	3.0	1.1	1.1	9.0.x	9.0.93	8 and later

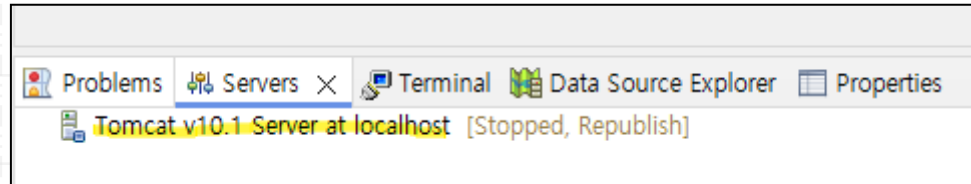
3. Tomcat 10 설치

Servlet6.0 및 JSP3.1 Tomcat10

나. 이클립스와 Tomcat 연동



다. Tomcat 설정 변경



The screenshot shows the 'Overview' configuration page for the Tomcat server. It is divided into several sections:

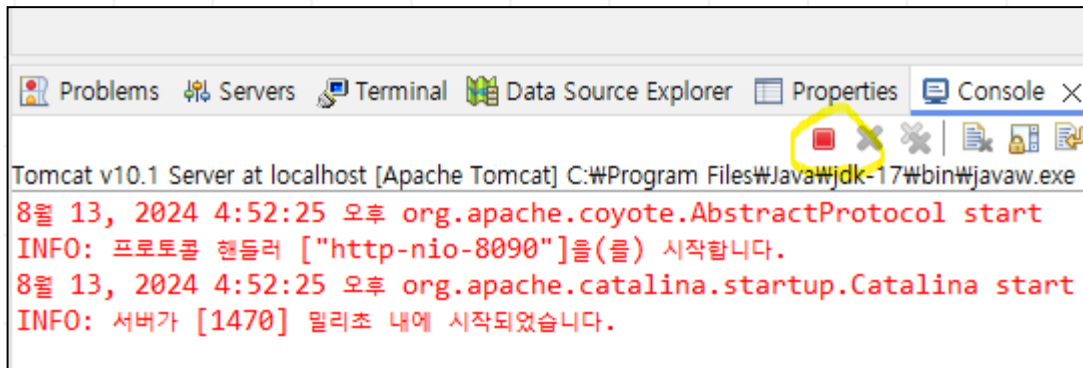
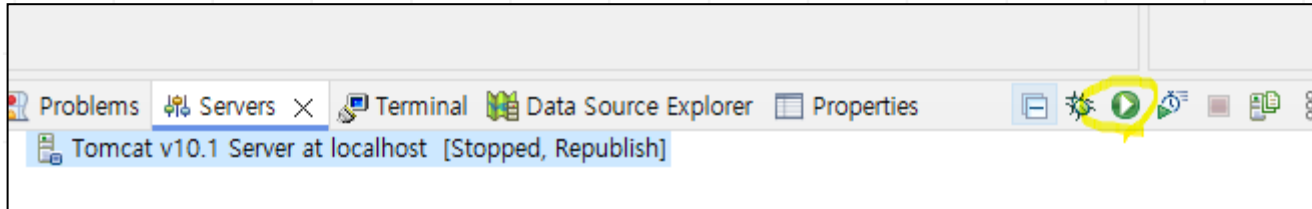
- General Information:** Contains fields for 'Server name' (Tomcat v10.1 Server at localhost), 'Host name' (localhost), and 'Runtime Environment' (Apache Tomcat v10.1). It also includes links for 'Configuration path' and 'Open launch configuration'.
- Server Locations:** Contains radio buttons for 'Use workspace metadata', 'Use Tomcat installation' (selected), and 'Use custom location'. It includes fields for 'Server path' and 'Deploy path', both with 'Browse...' buttons.
- Server Options:** Contains checkboxes for 'Serve modules without publishing', 'Publish module contexts to separate XML files', and 'Modules auto reload by default' (checked).
- Publishing:** A section header.
- Timeouts:** A section header.
- Ports:** A section header with a table to modify server ports.
- MIME Mappings:** A section header.

Port Name	Port Num
Tomcat admin port	8005
HTTP/1.1	8090

3. Tomcat 10 설치

Servlet6.0 및 JSP3.1 Tomcat10

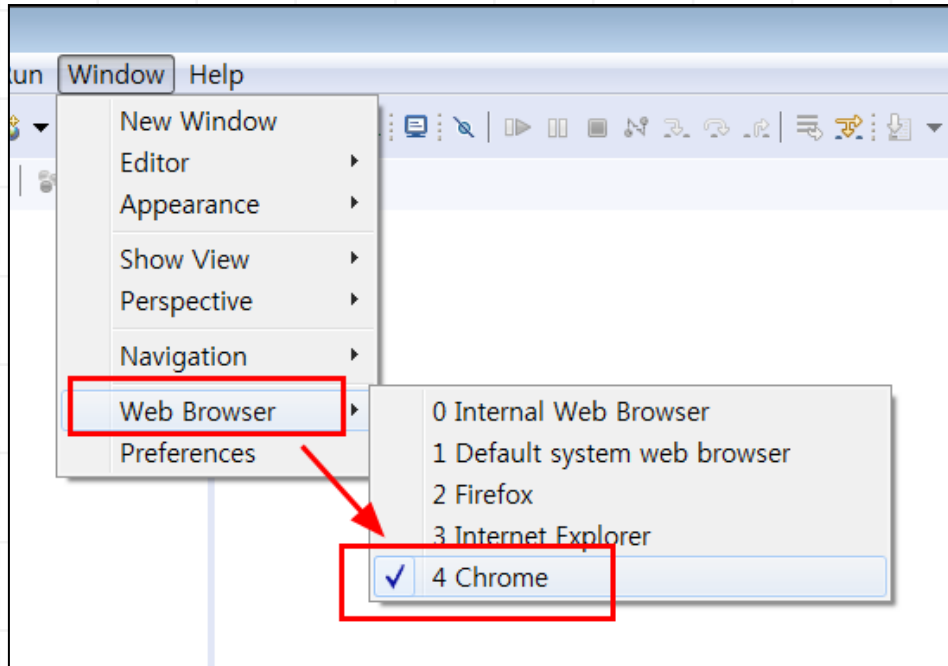
라. Tomcat 서버 시작 및 종료



3. Tomcat 10 설치

Servlet6.0 및 JSP3.1 Tomcat10

마. 기본 브라우저 변경

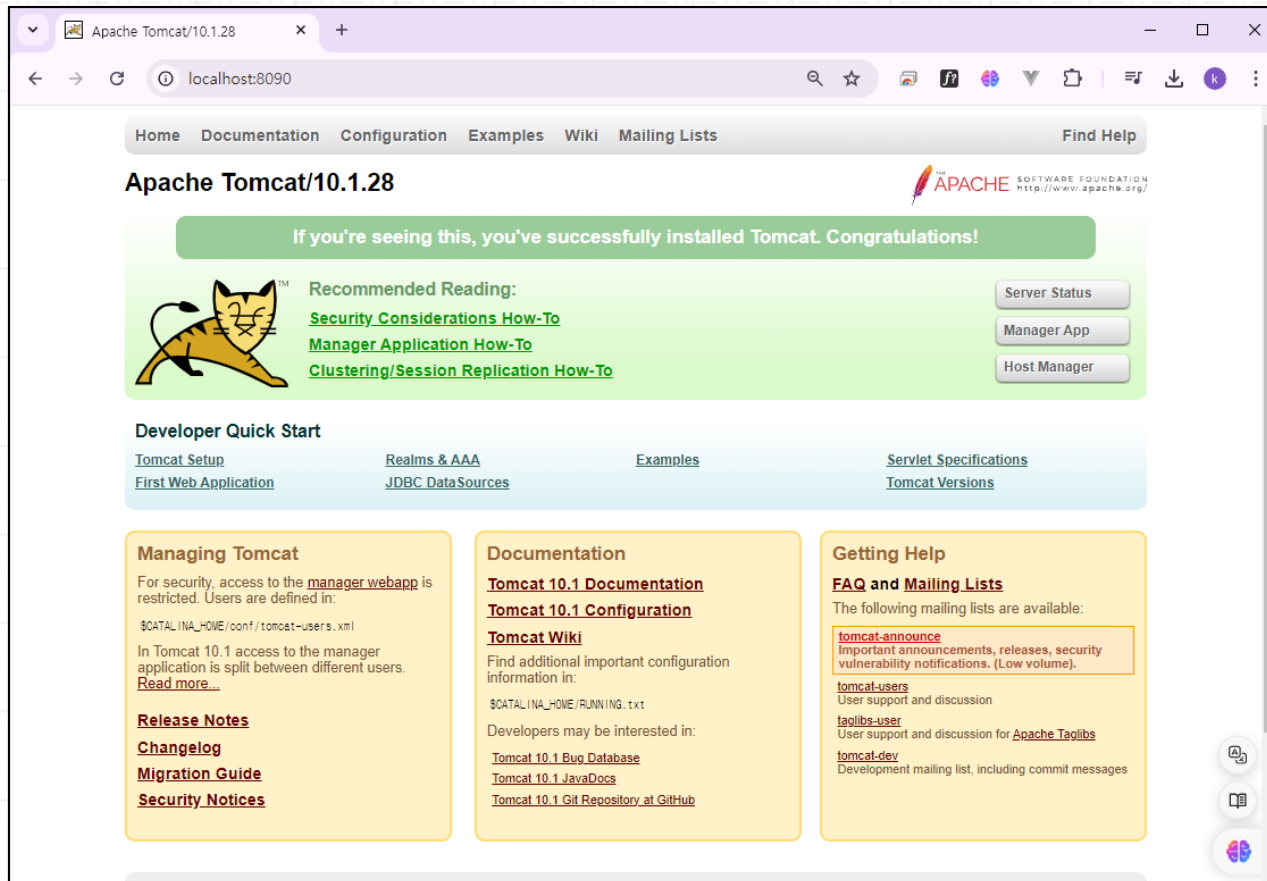


3. Tomcat 10 설치

Servlet6.0 및 JSP3.1 Tomcat10

바. 요청

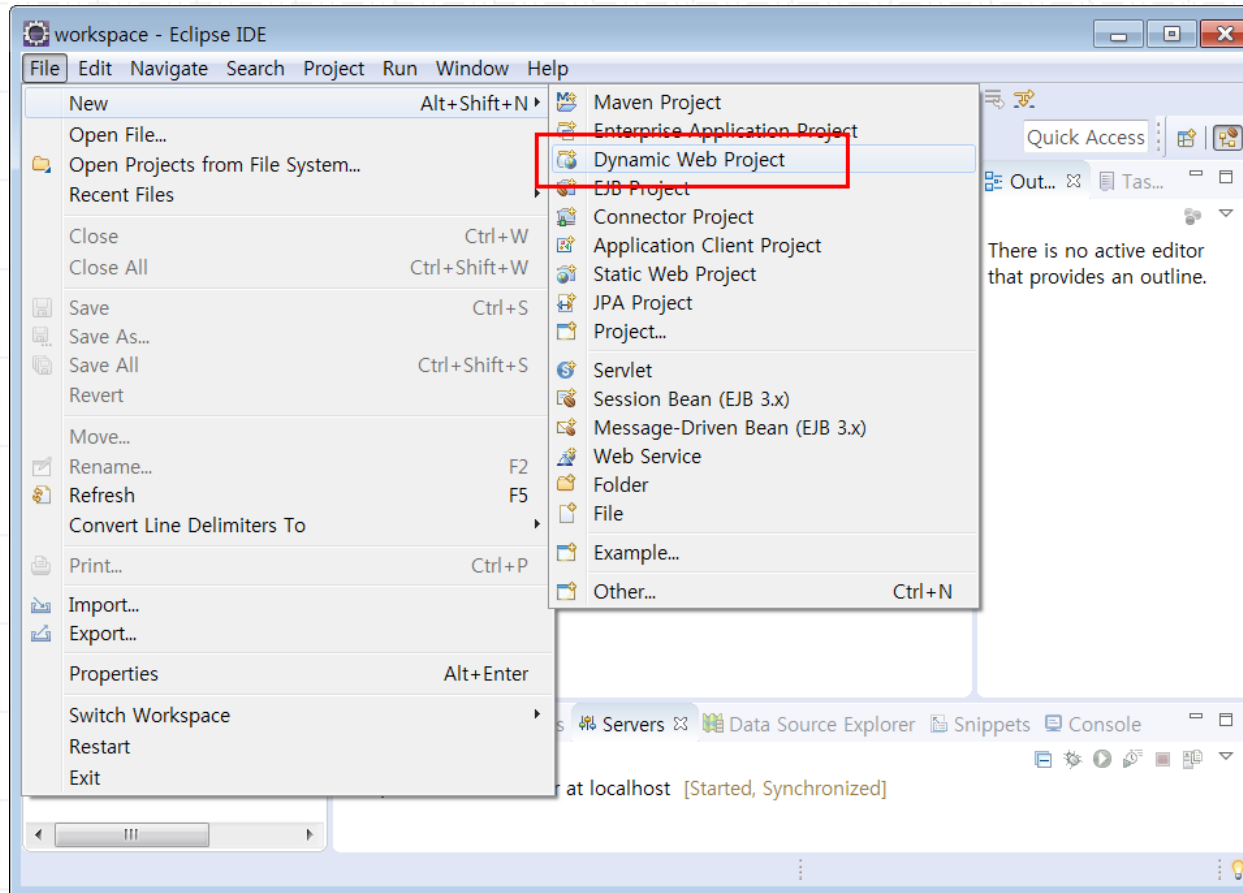
http://localhost:8090



4. 기본 코드 구성

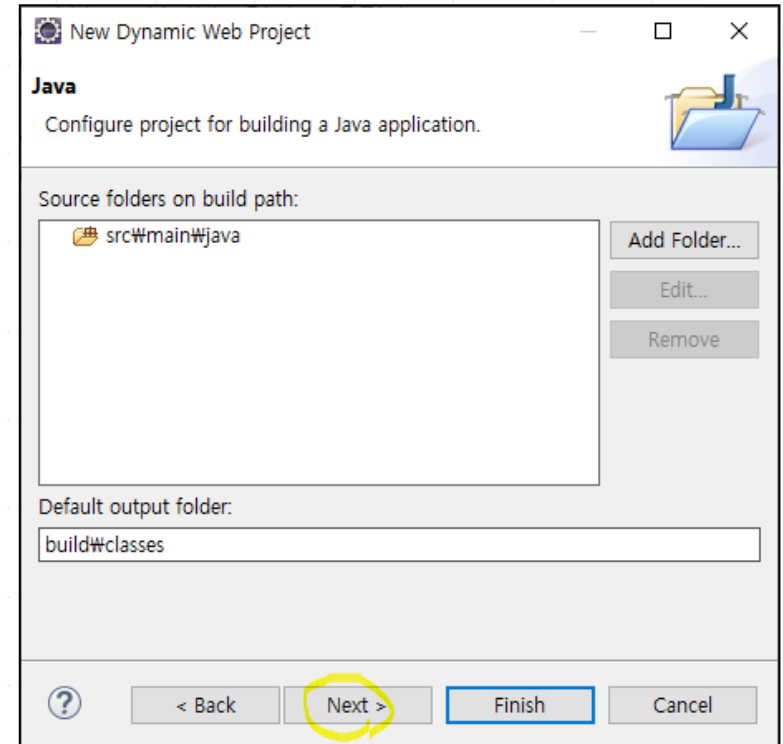
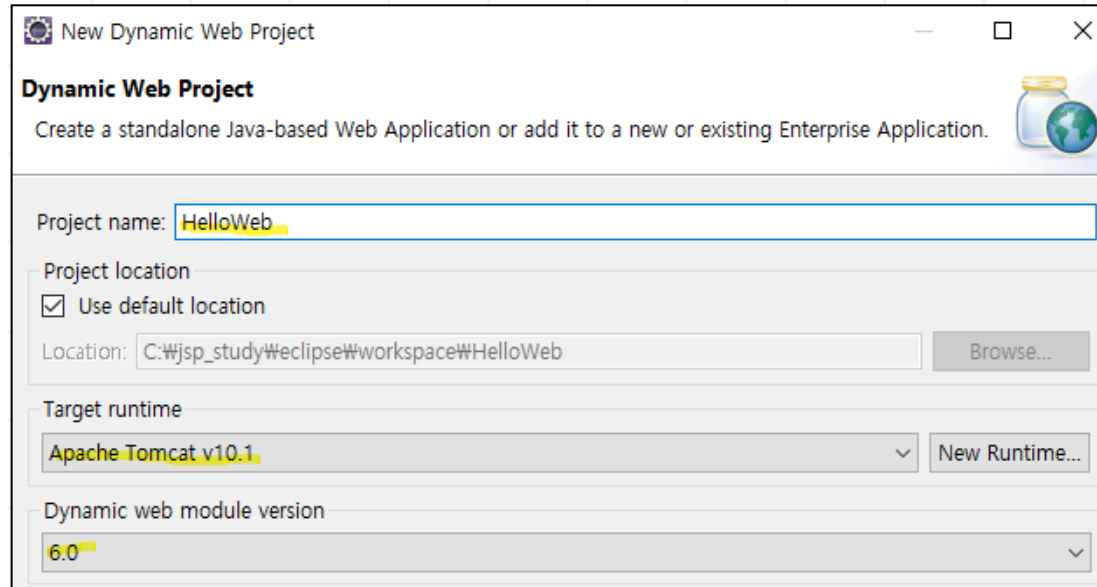
Servlet6.0 및 JSP3.1 Tomcat10

1) 프로젝트 생성 (Dynamic Web Project)



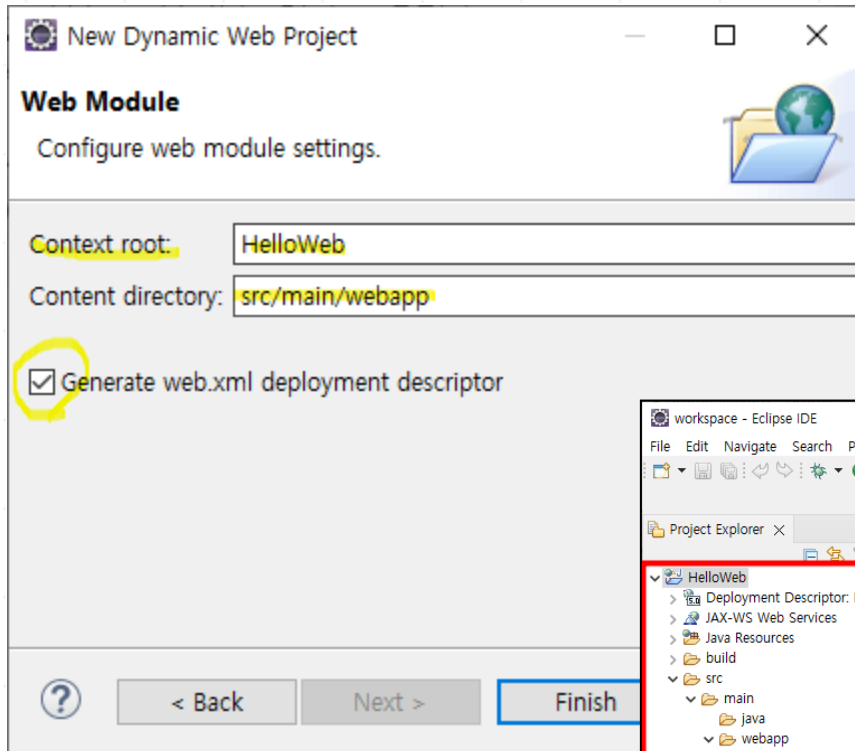
4. 기본 코드 구성

Servlet6.0 및 JSP3.1 Tomcat10

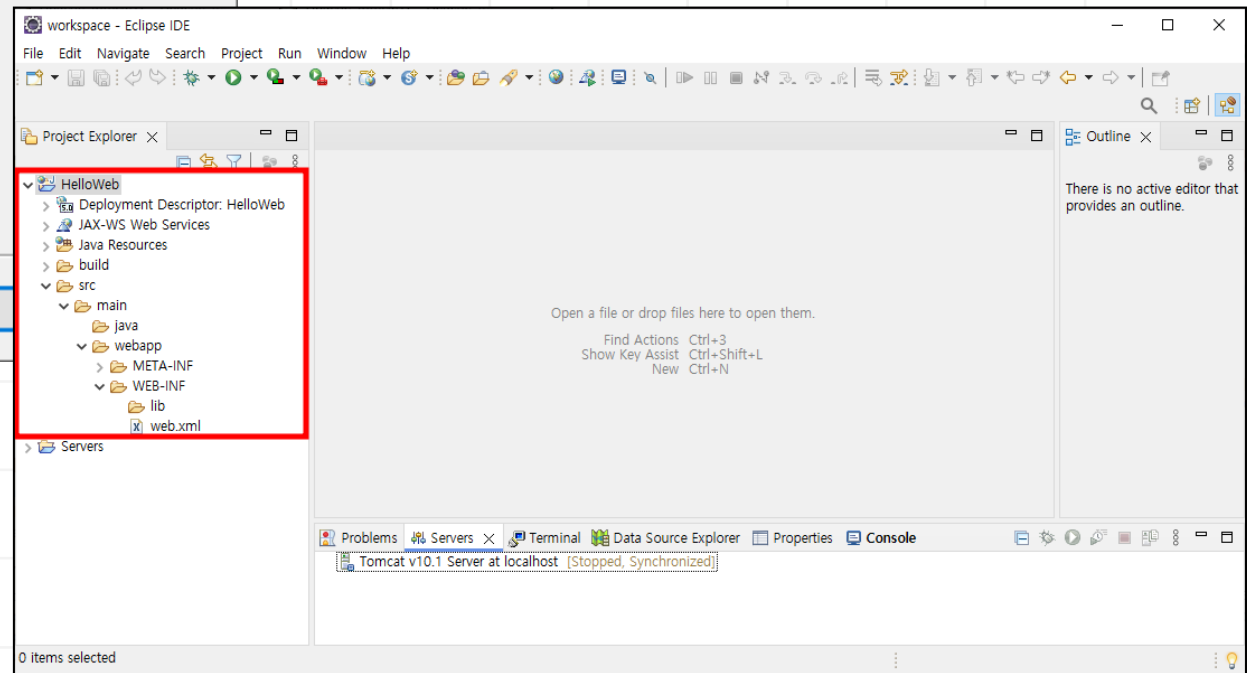


4. 기본 코드 구성

Servlet6.0 및 JSP3.1 Tomcat10



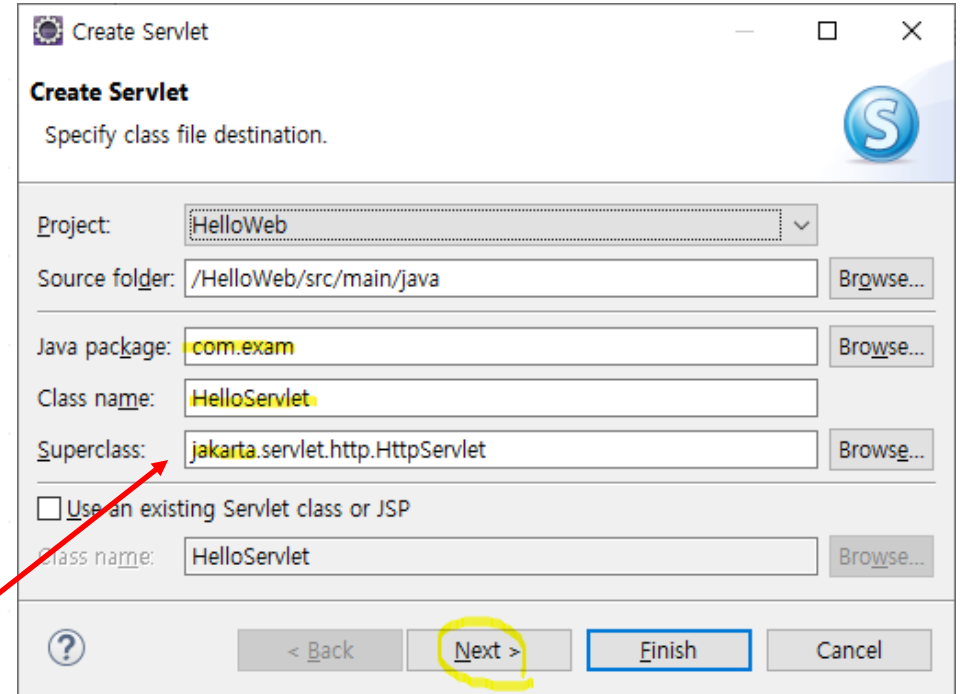
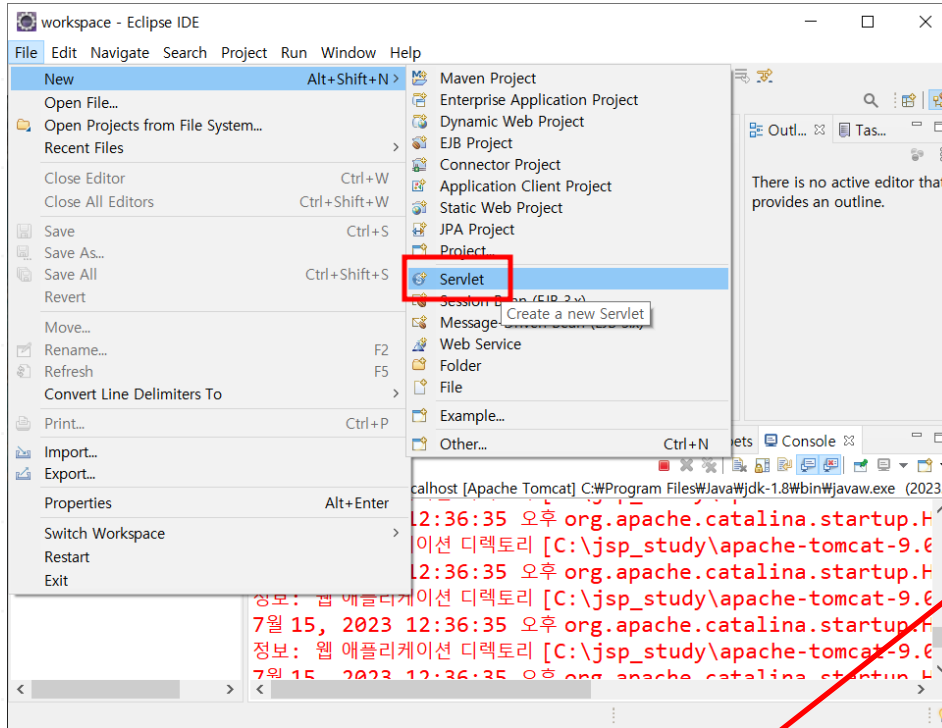
웹 컴포넌트(html,JSP,Servlet 및 이미지등)를 저장하는 물리적인 디렉터리와 웹 컨테이너가 논리적으로 관리하는 Context 이름을 지정하는 화면이다. 기본적으로 물리적인 디렉터리는 src/main/webapp로 자동으로 설정되고 Context명은 웹 프로젝트명과 일치한다.



4. 기본 코드 구성

Servlet6.0 및 JSP3.1 Tomcat10

2) 서블릿 생성



Tomcat 10 Software Downloads

Welcome to the Apache Tomcat® 10.x software download page. This page provides download links for obtaining the latest version of Tomcat 10.1.x software, as well as links to the archives of older releases.

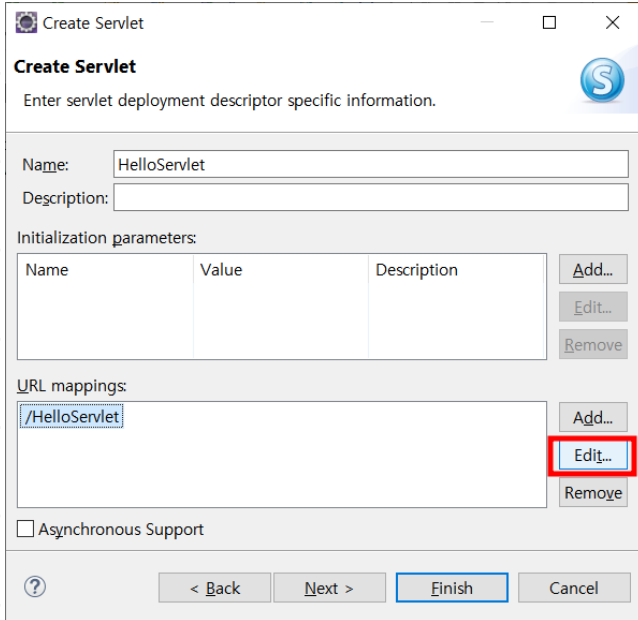
Unsure which version you need? Specification versions implemented, minimum Java version required and lots more useful information may be found on the ['which version?'](#) page.

Users of Tomcat 10 onwards should be aware that, as a result of the move from Java EE to Jakarta EE as part of the transfer of Java EE to the Eclipse Foundation, the primary package for all implemented APIs has changed from `javax.*` to `jakarta.*`. This will almost certainly require code changes to enable applications to migrate from Tomcat 9 and earlier to Tomcat 10 and later. A [migration tool](#) has been developed to aid this process.

4. 기본 코드 구성

Servlet6.0 및 JSP3.1 Tomcat10

3) 서블릿 �핑 (servlet mapping)



Create Servlet

Create Servlet

Enter servlet deployment descriptor specific information.

Name: HelloServlet

Description:

Initialization parameters:

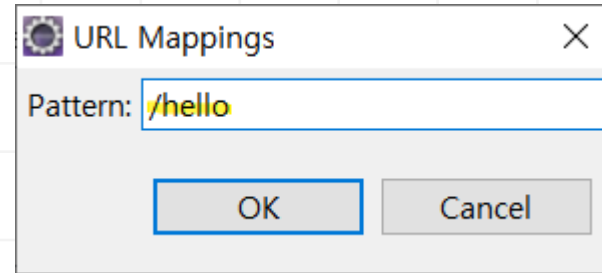
Name	Value	Description
------	-------	-------------

URL mappings:

/HelloServlet

Asynchronous Support

< Back Next > Finish Cancel



URL Mappings

Pattern: /hello

OK Cancel

OLD 방법

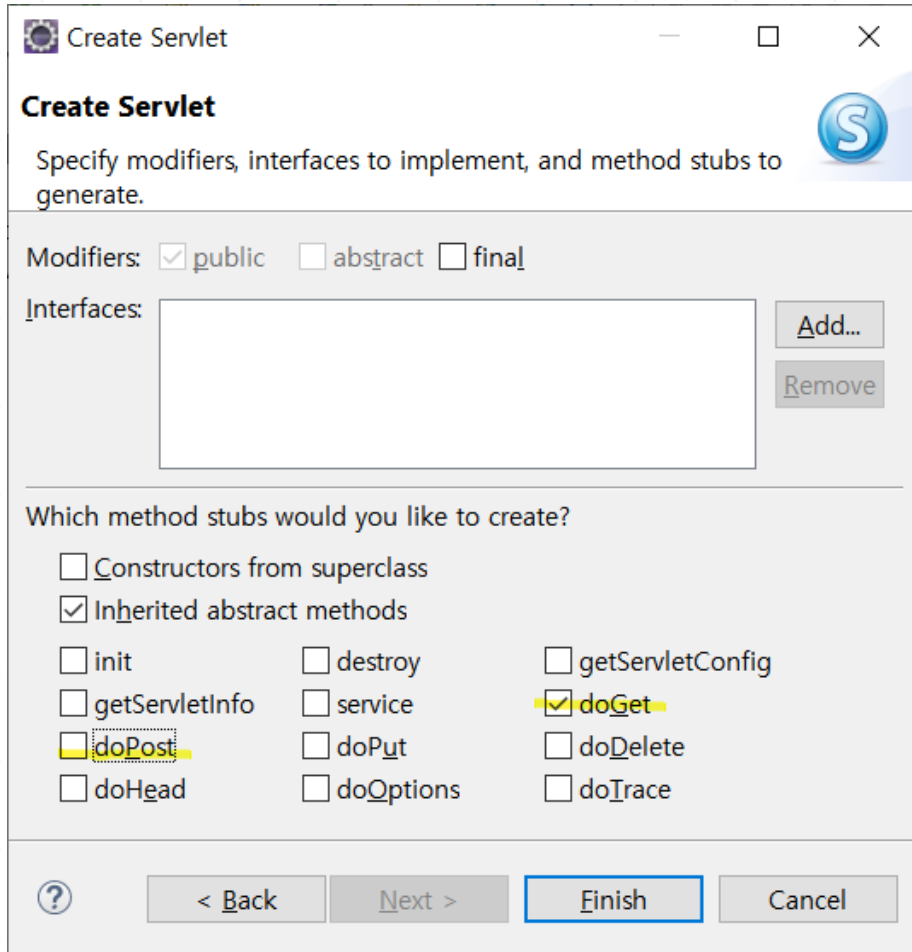
http://서버 IP 번호:포트 번호/컨텍스트명/servlet/패키지 포함한 서블릿명
예 > http://localhost:8090/HelloWeb/servlet/com.exam.HelloServlet



NEW 방법

http://서버 IP 번호:포트 번호/컨텍스트명/URL맵핑명
예 > http://localhost:8090/HelloWeb/hello

4) 서비스 메서드 구현



The image shows the 'Create Servlet' dialog box in an IDE. It has a title bar with a gear icon, the text 'Create Servlet', and standard window controls. The main area is titled 'Create Servlet' with a description: 'Specify modifiers, interfaces to implement, and method stubs to generate.' There is a blue 'S' icon in the top right. Under 'Modifiers', there are checkboxes for 'public' (checked), 'abstract', and 'final'. Under 'Interfaces', there is an empty text box and 'Add...' and 'Remove' buttons. A section titled 'Which method stubs would you like to create?' contains a list of checkboxes: 'Constructors from superclass' (unchecked), 'Inherited abstract methods' (checked), 'init' (unchecked), 'destroy' (unchecked), 'getServletConfig' (unchecked), 'getServletInfo' (unchecked), 'service' (unchecked), 'doGet' (checked and highlighted with a yellow box), 'doPost' (checked and highlighted with a yellow box), 'doPut' (unchecked), 'doDelete' (unchecked), 'doHead' (unchecked), 'doOptions' (unchecked), and 'doTrace' (unchecked). At the bottom, there is a help icon, and buttons for '< Back', 'Next >', 'Finish' (highlighted with a blue border), and 'Cancel'.

Create Servlet

Create Servlet

Specify modifiers, interfaces to implement, and method stubs to generate.

Modifiers: ☒ public ☐ abstract ☐ final

Interfaces: Add... Remove

Which method stubs would you like to create?

☐ Constructors from superclass

☒ Inherited abstract methods

☐ init ☐ destroy ☐ getServletConfig

☐ getServletInfo ☐ service ☒ doGet

☒ doPost ☐ doPut ☐ doDelete

☐ doHead ☐ doOptions ☐ doTrace

? < Back Next > Finish Cancel

4. 기본 코드 구성

Servlet6.0 및 JSP3.1 Tomcat10

5) 서블릿 구현 및 서블릿 매핑 설정.

```
HelloServlet.java x
1 package com.exam;
2
3 import jakarta.servlet.ServletException;
4 import jakarta.servlet.http.HttpServlet;
5 import jakarta.servlet.http.HttpServletRequest;
6 import jakarta.servlet.http.HttpServletResponse;
7 import java.io.IOException;
8
9
10 public class HelloServlet extends HttpServlet {
11
12     protected void doGet(HttpServletRequest request, HttpServletResponse response)
13         throws ServletException, IOException {
14
15         System.out.println("HelloServlet 요청");
16     }
17 }
18 }
19 }
```

```
web.xml x
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd (xsi:s
1 <?xml version="1.0" encoding="UTF-8"?>
2 <web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instan
3
4 <servlet> 서블릿 매핑
5     <servlet-name>HelloServlet</servlet-name>
6     <servlet-class>com.exam.HelloServlet</servlet-class>
7 </servlet>
8 <servlet-mapping>
9     <servlet-name>HelloServlet</servlet-name>
10    <url-pattern>/hello</url-pattern>
11 </servlet-mapping>
12 </web-app>
```

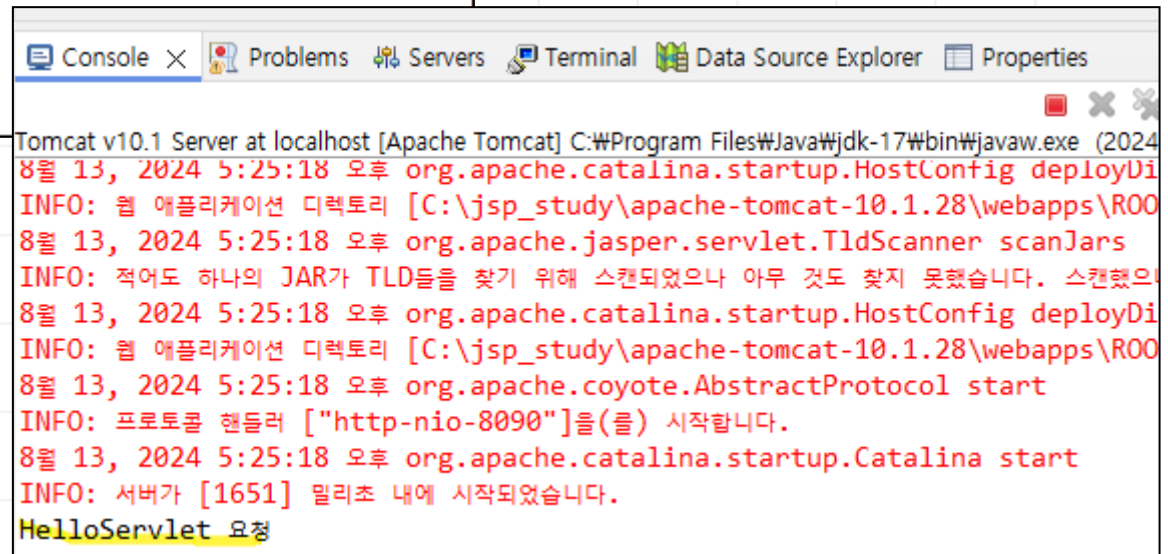
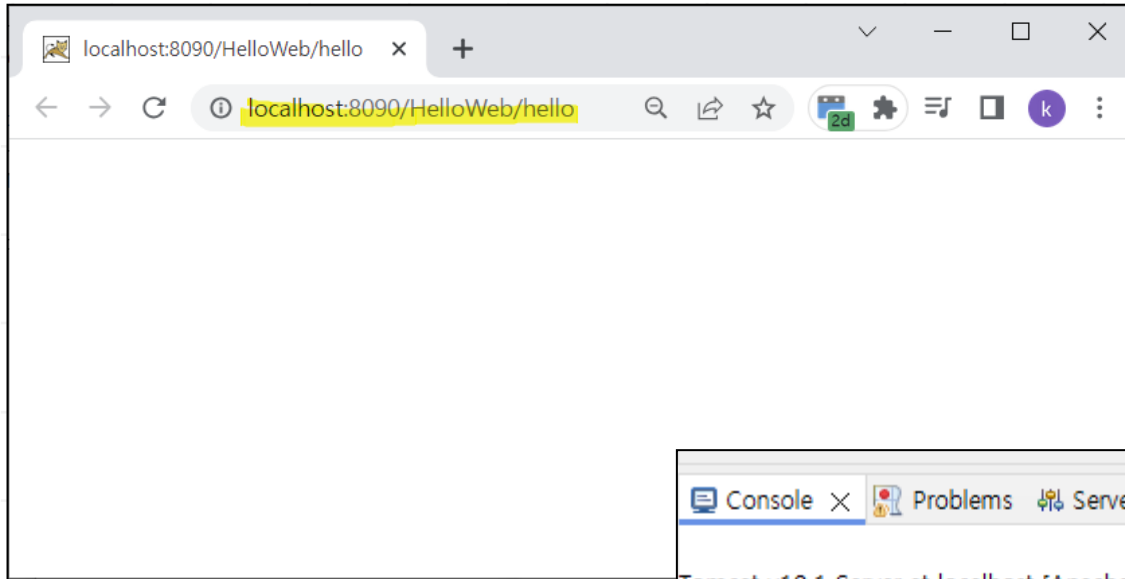

6) Tomcat 서버 시작 및 요청 (서블릿 선택 > 오른쪽 클릭 > Run As > .)

The image illustrates the process of running a servlet in Eclipse IDE. It consists of three main parts:

- Project Explorer:** Shows the project structure. The file `HelloServlet.java` under the `com.exam` package in the `src` folder is highlighted with a red rectangle.
- Run As Context Menu:** A right-click context menu is shown over the selected file. The `Run As` option is highlighted with a yellow circle. A sub-menu is visible, showing `1 Run on Server` (highlighted with a yellow circle) and `Run Configurations...`.
- Run On Server Dialog:** The `Run On Server` dialog box is shown. It prompts the user to select a server. Under the `localhost` group, `Tomcat v10.1 Server at localhost` is selected (highlighted with a yellow circle). The `Finish` button at the bottom is also highlighted with a yellow circle.

4. 기본 코드 구성

Servlet6.0 및 JSP3.1 Tomcat10



2장. 서블릿 (Servlet)

1. 서블릿 개요
2. 서블릿 구현 방법
3. 서블릿 맵핑
4. 서브릿 아키텍처
5. 응답 상태 코드
6. 서블릿 lifecycle 메서드

개요

서블릿은 웹 컨테이너에 의해서 관리(생성~소멸)되며 다양한 클라이언트 요청에 동적인 콘텐츠로 응답 가능한 자바 기반의 동적 웹 컴포넌트이다.

특징

클라이언트의 요청에 대해 동적으로 실행된다.
따라서 다양한 클라이언트 요구사항에 대한 반영이 가능하다.
main 메서드와 같은 시작점 역할의 메서드가 제공되지 않는다. 사용자가 요청한 서블릿이 시작점이 된다.
서블릿의 응답결과는 일반적으로 html 형식이기 때문에 html 코드로 응답하는 추가 작업이 필요하다. (MVC 패턴 적용하면 JSP가 담당)
MVC 패턴의 Controller 역할을 서블릿이 담당한다.
Tomcat 10 부터 javax 가 아닌 jakarta 로 패키지명이 변경됨.

1. 서블릿(Servlet)

Servlet6.0 및 JSP3.1 Tomcat10

구현방법

- 1) package 지정 필수
- 2) extends HttpServlet
- 3) 기본 생성자 필수
- 4) doGet(HttpServletRequest request, HttpServletResponse response)
또는
doPost(HttpServletRequest request, HttpServletResponse response)
- 5) 서블릿 맵핑 (@WebServlet 또는 web.xml 에서 설정가능)

샘플

```
package com.exam;

import java.io.IOException;

@WebServlet("/hello") web.xml 가능
public class HelloServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        System.out.println("HelloServlet 요청");
    }
}
```

개요

웹브라우저에서 서블릿을 요청하는 URL 값을 임의의 값으로 축약하는 작업으로서 보안 및 간소화의 장점을 제공한다.

전통적인 방법 (현재 사용 불가)

http://서버 IP 번호:포트 번호/컨텍스트명/servlet/패키지 포함한 서블릿명
예> http://localhost:8090/HelloWeb/servlet/com.exam.HelloServlet



서블릿 맵핑 방법

http://서버 IP 번호:포트 번호/컨텍스트명/서블릿맵핑
예> http://localhost:8090/HelloWeb/hello

2. 서블릿 맵핑

Servlet6.0 및 JSP3.1 Tomcat10

방법 2가지

@WebServlet 이용

```
@WebServlet("/hello")  
public class HelloServlet extends HttpServlet {
```

web.xml 이용

```
<servlet>  
  <servlet-name>helloServlet</servlet-name>  
  <servlet-class>com.exam.HelloServlet</servlet-class>  
</servlet>  
<servlet-mapping>  
  <servlet-name>helloServlet</servlet-name>  
  <url-pattern>/hello</url-pattern>  
</servlet-mapping>
```

Spring 프레임워크에서 사용하는 방식

web.xml은 웹 어플리케이션의 설정정보를 가진 xml 파일로서 WEB-INF 폴더에 저장되고 배치 지시자로 부른다.

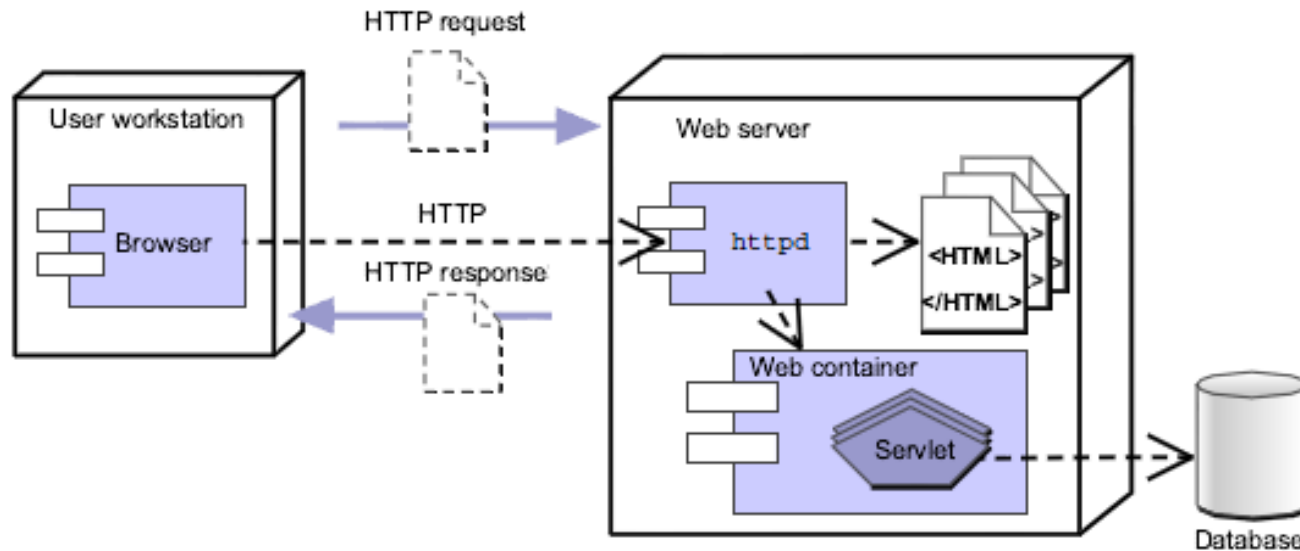
웹 컨테이너는 이 파일을 통해서 서블릿명과 맵핑명, 초기 파라미터, 필터 기능과 같은 정보를 파악할 수 있다.

3. 서블릿 아키텍처

Servlet6.0 및 JSP3.1 Tomcat10

개요

클라이언트에서 웹 브라우저를 이용하여 적절한 URL 형식으로 서블릿에 요청하면 웹 컨테이너에서 서블릿을 실행하고 결과값을 html로 작성하여 응답처리 한다.



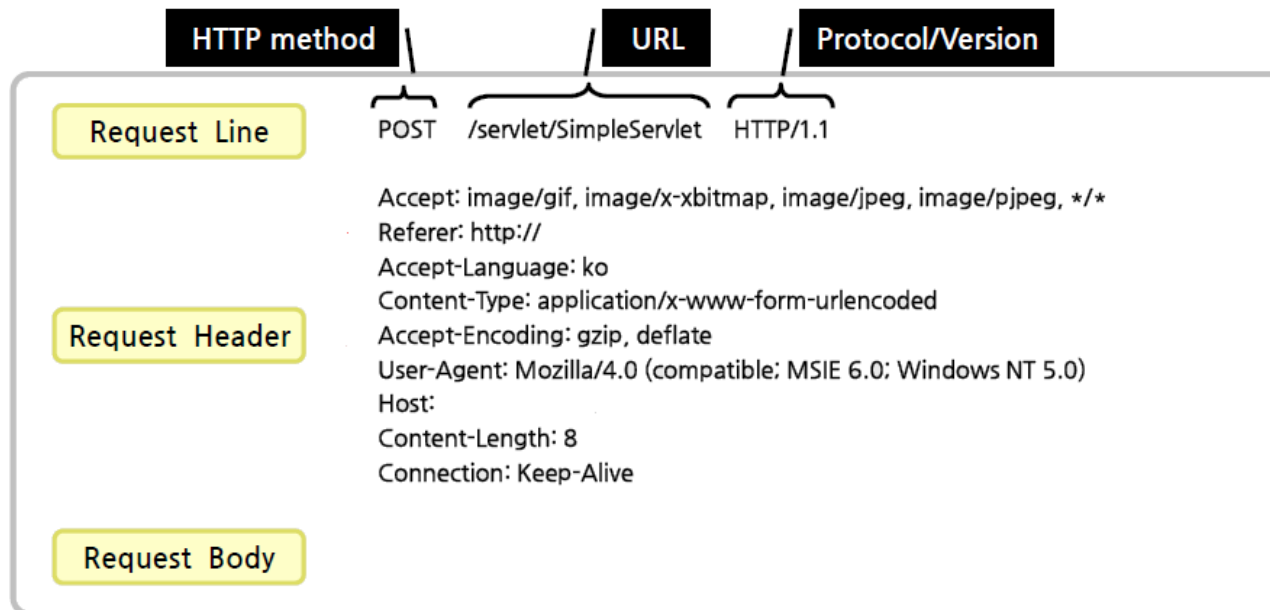
3. 서블릿 아키텍처

Servlet6.0 및 JSP3.1 Tomcat10

요청

HTTP 요청은 다음과 같이 3가지 요소로 구성된다.

- 1) 요청 라인 (request line) : method 방식, URL, 프로토콜 정보
- 2) 요청 헤더 (request header) : 브라우저 정보, 언어 및 인코딩, 서버 정보
- 3) 요청 본체 (request body) : 요청에 필요한 값 (폼 데이터등)

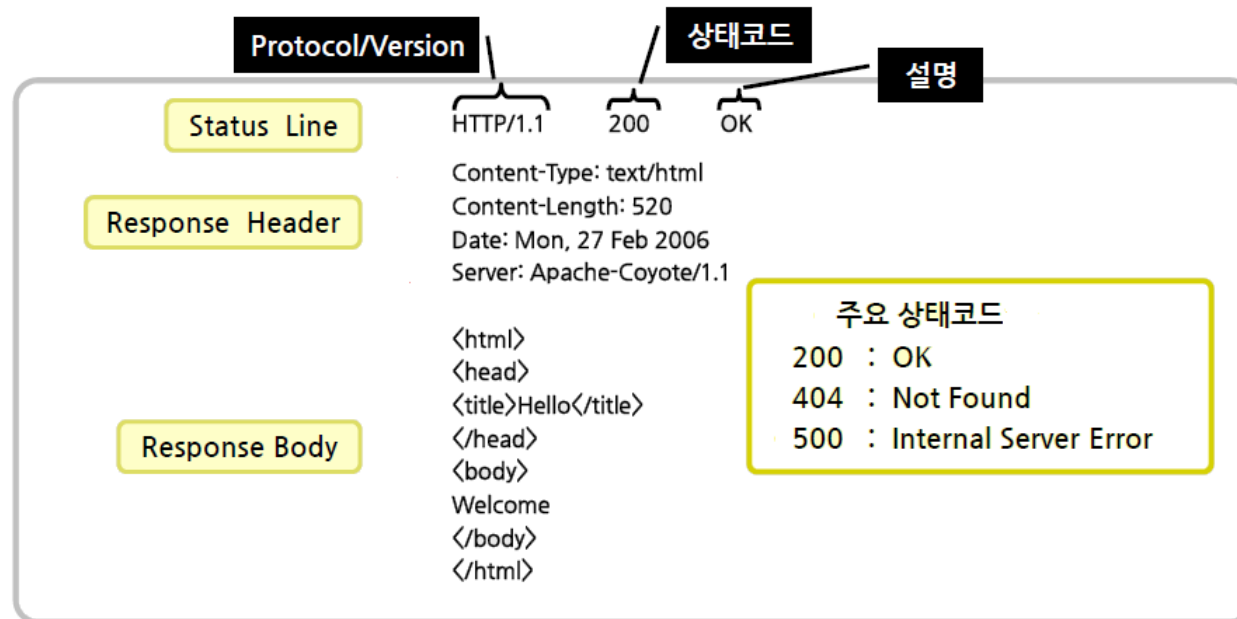


HttpServletRequest 객체가 요청처리를 담당한다.

응답

HTTP 응답은 다음과 같이 3가지 요소로 구성된다.

- 1) 상태 라인 (status line) : 응답상태코드, 프로토콜 정보
- 2) 응답 헤더 (response header) : 처리날짜, 인코딩, 서버정보등
- 3) 응답 본체 (response body) : 응답에 필요한 값 (html 문서)



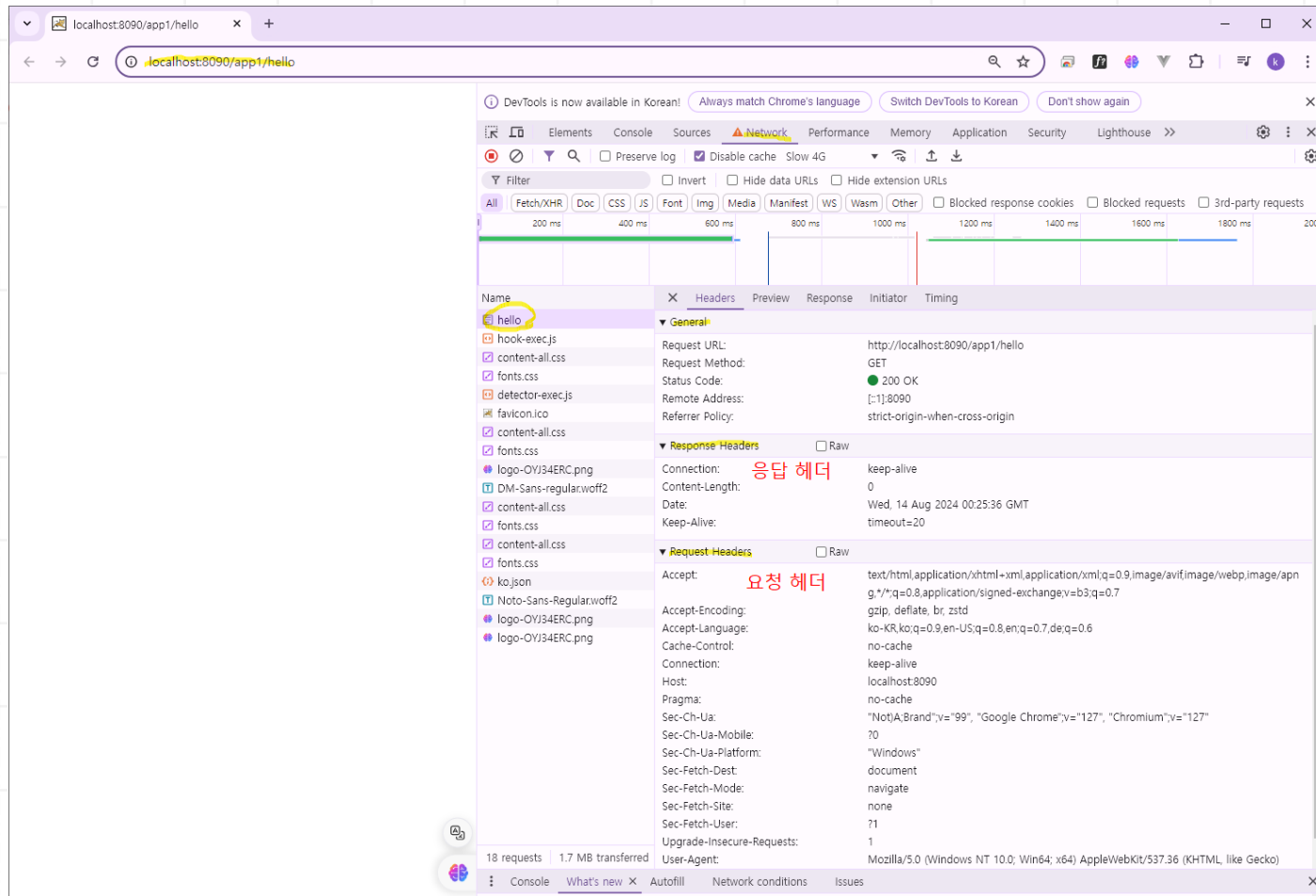
HttpServletResponse 객체가 응답처리를 담당한다.

3. 서블릿 아키텍처

Servlet6.0 및 JSP3.1 Tomcat10

크롬 웹브라우저에서 header 확인 (개발자도구 > Network)

http://localhost:8090/app1/hello



4. 응답 상태 코드 (status code)

Servlet6.0 및 JSP3.1 Tomcat10

응답 상태 코드

https://ko.wikipedia.org/wiki/HTTP_상태_코드

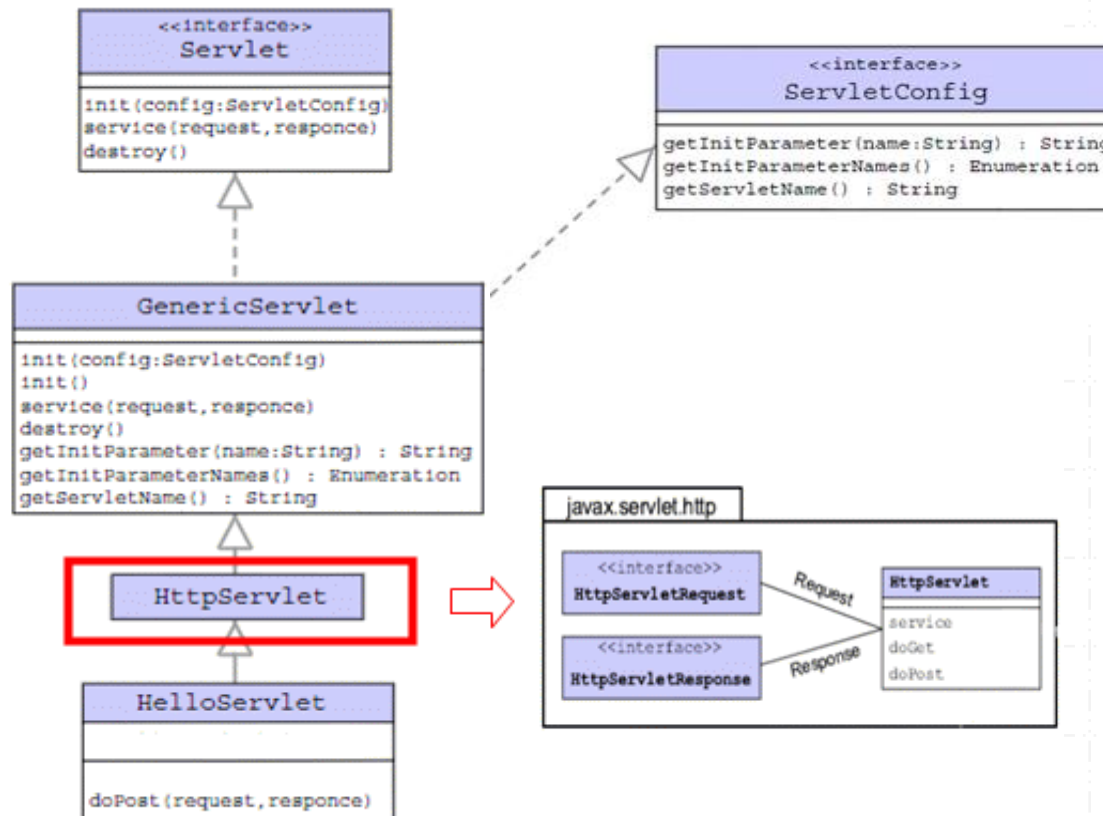
- 200(성공): 서버가 요청을 제대로 처리했다는 뜻이다. 이는 주로 서버가 요청한 페이지를 제공했다는 의미로 쓰인다.
- 403(Forbidden, 금지됨): 서버가 요청을 거부하고 있다. 예를 들자면, 사용자가 리소스에 대한 필요 권한을 갖고 있지 않다. (401은 인증 실패, 403은 인가 실패라고 볼 수 있음)
- 404(Not Found, 찾을 수 없음): 서버가 요청한 페이지(Resource)를 찾을 수 없다. 예를 들어 서버에 존재하지 않는 페이지에 대한 요청이 있을 경우 서버는 이 코드를 제공한다.
- 405(허용되지 않는 메소드): 요청에 지정된 방법을 사용할 수 없다. 예를 들어 POST 방식으로 요청을 받는 서버에 GET 요청을 보내는 경우, 또는 읽기 전용 리소스에 PUT 요청을 보내는 경우에 이 코드를 제공한다.
- 500(내부 서버 오류): 서버에 오류가 발생하여 요청을 수행할 수 없다.

5. HttpServlet API

Servlet6.0 및 JSP3.1 Tomcat10

개요

서블릿을 구현하기 위한 핵심 API로서 추상 클래스로 제공된다.
jakarta.servlet.http 패키지로 되어 있다.



5. HttpServlet API

Servlet6.0 및 JSP3.1 Tomcat10

Servlet 6.0 API 문서

<https://tomcat.apache.org/tomcat-10.1-doc/servletapi/index.html>

The screenshot shows the Apache Tomcat 10.1 documentation page for the Servlet API. The browser address bar displays tomcat.apache.org/tomcat-10.1-doc/index.html. The left sidebar contains a navigation menu with sections: Considerations, Reference, and Apache Tomcat Development. The main content area lists 35 numbered links for various configuration and development topics, including Apache Tomcat Clustering, Balancer, Connectors, Monitoring and Management, Logging, Apache Portable Runtime, Virtual Hosting, Advanced IO, Using Tomcat libraries with Maven, Security Considerations, Windows Service, Windows Authentication, High Concurrency JDBC Pool, WebSocket support, URL rewrite, CDI and JAX-RS support, and AOT compilation support. A 'Reference' section at the bottom states that the following documents are aimed at System Administrators who are responsible for the configuration and operation of the server.

19. [Apache Tomcat Clustering](#) - Enable session replication in a Apache Tomcat cluster.

20. [Balancer](#) - Configuring, using, and extending the load balancer available in Apache Tomcat.

21. [Connectors](#) - Connectors available in Apache Tomcat, and native connectors.

22. [Monitoring and Management](#) - Enabling JMX Remote support, and monitoring Apache Tomcat.

23. [Logging](#) - Configuring logging in Apache Tomcat.

24. [Apache Portable Runtime](#) - Using APR to provide access to Operating System native functions.

25. [Virtual Hosting](#) - Configuring virtual hosting in Apache Tomcat.

26. [Advanced IO](#) - Extensions available over regular, blocking IO.

27. [Using Tomcat libraries with Maven](#) - Obtaining Tomcat jars through Maven.

28. [Security Considerations](#) - Options to consider when securing an Apache Tomcat installation.

29. [Windows Service](#) - Running Tomcat as a service on Microsoft Windows.

30. [Windows Authentication](#) - Configuring Tomcat to use integrated Windows authentication.

31. [High Concurrency JDBC Pool](#) - Configuring Tomcat to use an alternative JDBC pool.

32. [WebSocket support](#) - Developing WebSocket applications for Apache Tomcat.

33. [URL rewrite](#) - Using the regexp based rewrite valve for conditional rewriting.

34. [CDI and JAX-RS support](#) - Configuring CDI, JAX-RS and Eclipse MicroProfile.

35. [AOT compilation support](#) - Ahead of Time compilation support for Java 11.

Reference

The following documents are aimed at *System Administrators* who are responsible for the configuration and operation of the server.

- [Release notes](#) - Known issues in this Apache Tomcat release.
- [Apache Tomcat Server Configuration Reference](#) - Reference manual for the `conf/server.xml` file.
- [JK Documentation](#) - Complete documentation and HOWTOs on the use of the `mod_jk` module.
- Servlet 6.0 [Specification](#) and [Javadoc](#)
- JSP 3.1 [Specification](#) and [Javadoc](#)
- EL 5.0 [Specification](#) and [Javadoc](#)
- WebSocket 2.1 [Specification](#) and [Javadoc](#)
- Authentication 2.0 [Specification](#) and [Javadoc](#)

Apache Tomcat Developers

The screenshot shows the documentation for the `HttpServlet` class. It is part of the `jakarta.servlet.http` package. The class is an abstract class that extends `GenericServlet`. It implements the `Servlet`, `ServletConfig`, and `Serializable` interfaces. The documentation provides a brief description of the class and a list of methods that must be implemented by subclasses.

Package `jakarta.servlet.http`

Class `HttpServlet`

`java.lang.Object`
`jakarta.servlet.GenericServlet`
`jakarta.servlet.http.HttpServlet`

All Implemented Interfaces:
`Servlet`, `ServletConfig`, `Serializable`

public abstract class **HttpServlet**
extends `GenericServlet`

Provides an abstract class to be subclassed to create an HTTP servlet suitable for a Web container.

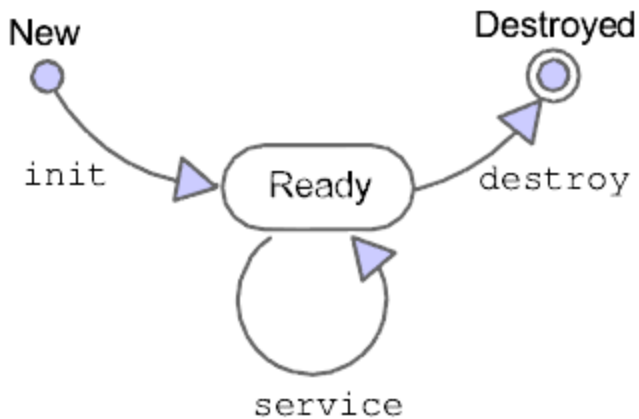
- `doGet`, if the servlet supports HTTP GET requests
- `doPost`, for HTTP POST requests
- `doPut`, for HTTP PUT requests
- `doDelete`, for HTTP DELETE requests
- `init` and `destroy`, to manage resources that are held for the life of the servlet
- `getServletInfo`, which the servlet uses to provide information about itself

6. Lifecycle 메서드

Servlet6.0 및 JSP3.1 Tomcat10

개요

Tomcat 컨테이너는 서블릿의 인스턴스를 `init`, `service(doGet/doPost)`, `destroy`의 3가지 메서드를 사용하여 관리한다.



6. Lifecycle 메서드

Servlet6.0 및 JSP3.1 Tomcat10

init 메서드

웹 컨테이너에 의해서 서블릿 인스턴스가 처음 생성될 때 호출되고 어플리케이션 전체 구간에서 단 한번만 실행된다.

호출 시점은 브라우저에서 처음 요청할 때이며 서블릿의 초기화 작업시 주로 사용된다.

또한 단 한번만 실행된다는 것은 **서블릿이 단 한번만 생성됨을 의미**하고 하나의 서블릿을 사용하여 다수의 클라이언트 요청을 처리하게 된다.

service 메서드 (doGet,doPost)

클라이언트가 원하는 동적 처리 작업을 담당하는 메서드로서 요청할 때마다 실행된다. 메서드의 파라미터로는 요청을 처리하는 `HttpServletRequest` 객체와 응답을 처리하는 `HttpServletResponse` 객체를 사용한다.

destroy 메서드

서블릿 인스턴스가 웹 컨테이너에서 제거될 때 호출된다.

init 메서드와는 반대로 사용했던 자원을 반납 처리하는 코드 구현시 사용된다.

6. Lifecycle 메서드

Servlet6.0 및 JSP3.1 Tomcat10

샘플

```
@WebServlet("/TestServlet")
public class TestServlet extends HttpServlet {

    public void init(ServletConfig config) throws ServletException {
        System.out.println("init 메서드");
    }

    public void destroy() {
        System.out.println("destroy 메서드");
    }

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        System.out.println("doGet 메서드");
    }
}
```

정보: 서버가 [774] 밀리초 내에 시작되었습니다.

init 메서드

doGet 메서드



정보: 서버가 [774] 밀리초 내에 시작되었습니다.

init 메서드

doGet 메서드

doGet 메서드

doGet 메서드

7월 15, 2023 4:58:44 오후 org.apache.c

정보: 이름이 [/app]인 컨텍스트를 다시 로드하는 작

destroy 메서드 서블릿 수정시 destroy 확인 가능

7. thread-safe vs thread-unsafe

Servlet6.0 및 JSP3.1 Tomcat10

개요

서블릿은 단 한번만 생성되기 때문에 인스턴스 변수도 단 한번만 생성된다.
따라서 여러 클라이언트가 공유할 수 있다. (thread-unsafe)
만약에 여러 클라이언트와 공유하지 않고 자신만의 데이터를 유지하고자 한다면
반드시 로컬 변수로 작성해야 된다. (thread-safe)

```
@WebServlet("/TestServlet")
public class TestServlet extends HttpServlet {

    //인스턴스 변수, static 변수 ==> 공유가능(thread-unsafe)
    int num;
    ArrayList<String> list = new ArrayList<String>();

    protected void doGet(HttpServletRequest request, HttpServletResponse response)

        //인스턴스변수==> 공유가능==>thread-unsafe
        num++;
        list.add("홍길동");

        //로컬변수==> 공유불가==>thread-safe
        int size =10;

        System.out.println("doGet:num >>" + num);
        System.out.println("doGet:list >>" + list);
        System.out.println("doGet:size >>" + size);
    }
```

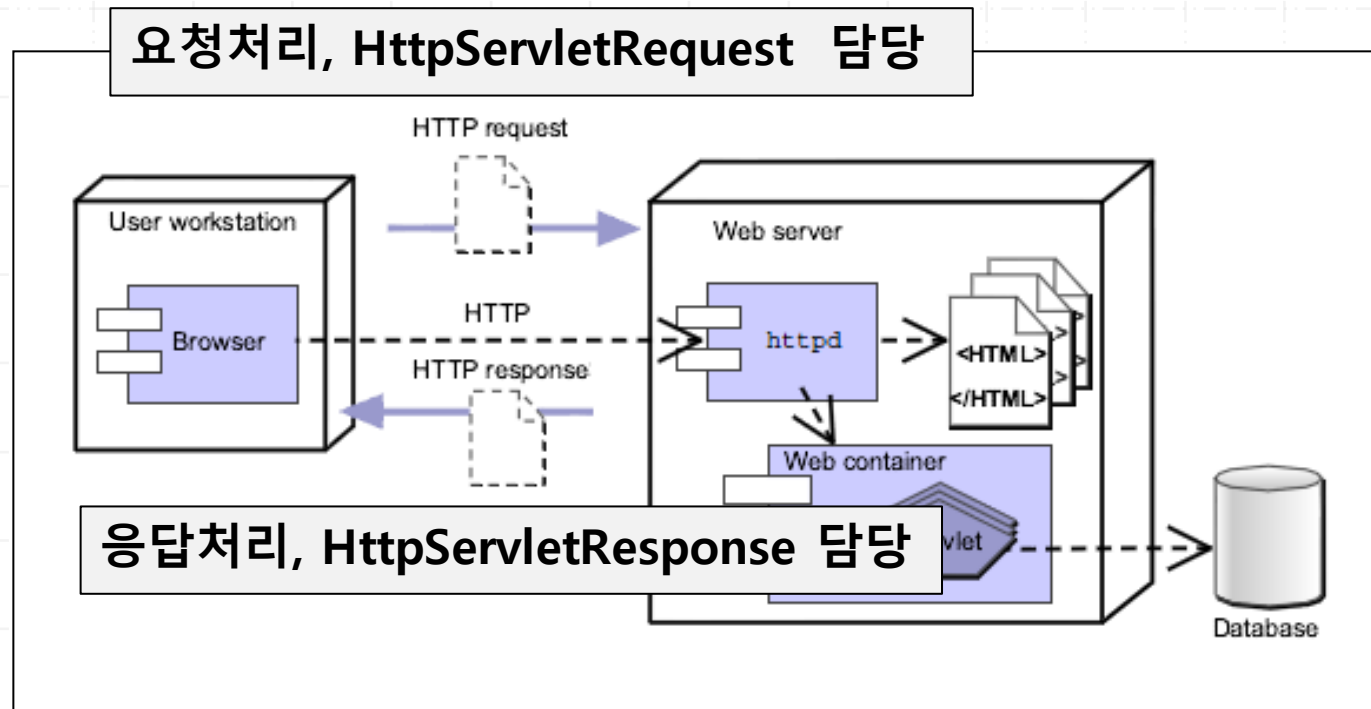
3장. 요청 및 응답 처리

1. 요청 및 응답처리
2. 서블릿 응답처리
3. 서블릿 요청처리
4. POST 방식의 한글처리

1. 요청 및 응답처리

Servlet6.0 및 JSP3.1 Tomcat10

개요



2. 서블릿 응답처리

Servlet6.0 및 JSP3.1 Tomcat10

개요

클라이언트에서 서블릿으로 요청을 하면 서블릿은 실행한 결과를 html 형식으로 작성하여 클라이언트에게 전송(응답처리)한다.

이러한 응답처리를 담당하는 API는 HttpServletResponse 객체이고 MIME 타입 및 자바의 I/O를 사용하여 응답처리를 구현할 수 있다.

(MVC 패턴 적용시에는 응답처리는 JSP가 담당한다.)

가. MIME 타입 지정 (Multipurpose Internet Mail Extensions)

서버는 클라이언트에게 처리할 데이터의 종류를 MIME 타입을 사용하여 알려준다. 기본 MIME 타입은 일반 텍스트를 의미하는 text/plain이고 html 전송은 text/html로 지정한다. 한글을 사용하기 위해서는 text/html;charset=utf-8 형식으로 사용해야 된다.

문법:

```
response.setContentType("text/html;charset=utf-8");
```

2. 서블릿 응답처리

Servlet6.0 및 JSP3.1 Tomcat10

나. 출력 스트림 생성

서블릿 및 JSP을 활용한 응답처리는 기본적으로 자바I/O 기술을 이용한다. 따라서 자바의 출력 스트림을 위한 OutputStream 객체 또는 Writer 객체를 사용해야 된다. 서블릿에서는 getWriter() 메서드를 이용한 PrintWriter 객체와 getOutputStream() 메서드를 이용한 ServletOutputStream 객체를 사용할 수 있다.

문법:

```
response.setContentType("text/html;charset=utf-8");
```

```
PrintWriter out = response.getWriter();
```

```
out.print("<html> <body>");
```

```
out.print("hello, 안녕하세요");
```

```
out.print("</body> </html>");
```

2. 서블릿 응답처리

Servlet6.0 및 JSP3.1 Tomcat10

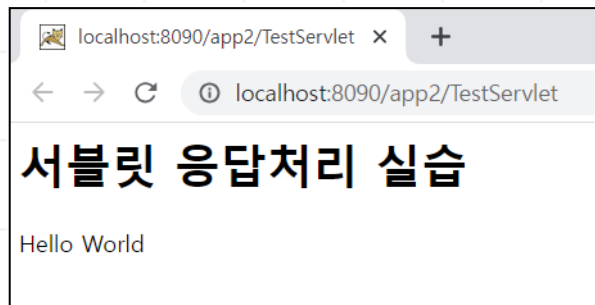
샘플

```
@WebServlet("/TestServlet")
public class TestServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.print("<html><body>");
        out.print("<h1>서블릿 응답처리 실습</h1>");
        out.print("<p>Hello World</p>");
        out.print("</body></html>");

    }
}
```



개요

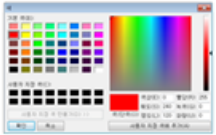
html 태그의 폼은 Servlet/JSP을 포함한 웹 어플리케이션을 개발할 때 반드시 필요한 기본이 되는 요소이다. 클라이언트와 서버간 데이터 전달을 가능하게 하는 다양한 태그들을 제공하며 회원가입, 로그인 등과 같이 사용자가 직접 데이터를 입력할 수 있도록 지원한다.

종류

홍길동	<code><input type="text" name="username" value="홍길동"></code>
1234	<code><input type="password" name="passwd" value="1234"></code>
<input type="file"/> 찾아보기...	<code><input type="file" name="theFile" ></code>
☐ value1 ☒ value2 ☒ value3	<code><input type="checkbox" name="check" value='value1' ></code> <code><input type="checkbox" name="check" value='value2' ></code> <code><input type="checkbox" name="check" value='value3' ></code>

3. html 폼 태그

Servlet6.0 및 JSP3.1 Tomcat10

☐ value1 ☒ value2 ☐ value3	<pre><input type="radio" name="check" value='value1' > <input type="radio" name="check" value='value2' > <input type="radio" name="check" value='value3' ></pre>	Volvo ▼ Volvo Saab Fiat Audi	<pre><select name="cars"> <option value="volvo">Volvo</option> <option value="saab">Saab</option> <option value="fiat">Fiat</option> <option value="audi">Audi</option> </select></pre>
일반단추	<pre><input type="button" name="button" value="일반 단추" ></pre>		
멀티라인 	<pre><textarea name="content" rows="3" cols="20"> </textarea></pre>		
보내기단추	<pre><input type="submit" value="보내기단추" ></pre>		
	<pre><input type="color" name="favcolor" value="#ff0000"></pre>		
연도 - 월 - 일	<pre><input type="date" name="bday"></pre>		
E-mail: inky4832@hanmail.net	<pre><input type="email" name="email"></pre>		
2	<pre><input type="number" name="quantity" min="1" max="5"></pre>		
Points: <input type="range" value="5"/>	<pre><input type="range" name="points" min="0" max="10"></pre>		
Search Google: 맛집	<pre><input type="search" name="googlesearch"></pre>		

form 태그와 submit

form 태그는 여러가지 사용자 입력 태그들을 포함하는 컨테이너(container) 역할을 담당하는 태그이다. 서버에 사용자가 입력한 데이터를 전송하기 위해서는 반드시 form 태그내에 input 태그들을 지정해야 된다.

form 태그내에 지정하지 않은 input 태그는 서버로 전달되지 않는다.

submit 태그는 서버에 사용자가 입력한 데이터를 전송하기 위한 버튼이다.

action 속성

submit 버튼을 선택했을 때 서버에서 요청을 처리할 웹 컴포넌트를 지정한다.

예>

```
<form action="login.html">
```

```
<form action="login.jsp">
```

```
<form action="login"> // login으로 맵핑된 서블릿
```

method 속성- GET

기본처리 방법으로서 요청 파라미터 값이 웹 브라우저의 URL에 명시되어 전달된다.
(정확히 설명하면 HTTP 요청의 request line에 포함되어 전송됨.)

클라이언트에서 GET 방식으로 요청하면 서버에서는 doGet 메서드가 처리한다.
전달되는 파라미터 길이에 제한이 있으며 URL에 명시적으로 전달되기 때문에 보안에 취약하다. 웹의 링크(link) 및 명시적인 URL 요청등은 모두 GET 방식으로 요청하는 형태이다. 한글처리는 기본적으로 지원된다.

예> http://서버IP번호:포트번호/컨텍스트명/경로명/login?name=홍길동&age=20

method 속성- POST

요청 파라미터 값이 URL이 아닌 HTTP 요청의 request body에 포함되어 전송된다.
클라이언트에서 POST 방식으로 요청하면 서버에서는 doPost 메서드가 처리한다.
GET 방식과 비교해서 보안에 덜 취약하고 새로고침시 재요청되지 않고 사용자에게 재요청 의사를 묻는 정보창이 보여진다.

한글처리는 request.setCharacterEncoding("utf-8") 메서드를 사용해야 된다.

예> url : http://서버IP번호:포트번호/컨텍스트명/경로명/login
request body : name=홍길동&age=20

요청 파라미터 처리

html의 사용자 입력 태그에 값을 지정하고 submit 버튼을 선택하면 서버에 파라미터가 전달된다. 요청 받은 서블릿은 `HttpServletRequest` 객체의 메서드를 사용하여 요청 파라미터 값을 얻을 수 있다.

주요 메서드

리턴 타입	메서드명	내용
String	<code>getParameter(name)</code>	<code>name</code> 에 해당되는 파라미터 값을 리턴한다. 만약 지정된 <code>name</code> 의 파라미터 값이 없으면 <code>null</code> 을 리턴한다.
String []	<code>getParameterValues(name)</code>	<code>checkbox</code> , <code>radio</code> 태그와 같이 하나의 <code>name</code> 에 여러 값을 가지는 경우에 주로 사용된다. <code>name</code> 에 해당되는 파라미터 값을 배열로 리턴한다.
Enumeration	<code>getParameterNames()</code>	폼 태그 안에 여러 개의 <code>input</code> 태그가 있는 경우에 주로 사용된다. 모든 <code>name</code> 값을 Enumeration 타입으로 리턴한다. 얻은 <code>name</code> 값을 이용하여 파라미터 값을 얻는다.

실습

- 아이디
- 비밀번호
-

```
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    String userid = request.getParameter("userid");
    String passwd = request.getParameter("passwd");

    response.setContentType("text/html; charset=utf-8");

    PrintWriter out = response.getWriter();
    out.print("<html><body>");
    out.print("아이디값: " + userid + "<br>");
    out.print("비밀번호값: " + passwd + "<br>");
    out.print("</body></html>");

}
```

실습

- 야구 ☐ 축구 ☒ 농구 ☒
- 남 ☒ 여 ☐
- 전송

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    request.setCharacterEncoding("utf-8");

    String [] sports = request.getParameterValues("sports");
    String gender = request.getParameter("gender");

    response.setContentType("text/html; charset=utf-8");
    PrintWriter out = response.getWriter();
    out.print("<html><body>");
    for (String sport : sports) {
        out.print("좋아하는 운동2: " + sport + "<br>");
    }
    out.print("성별 : " + gender + "<br>");
    out.print("</body></html>");

}
```

5. 경로(path)

Servlet6.0 및 JSP3.1 Tomcat10

개요

특정 타겟(서블릿,jsp,html,css파일,js파일, image등)의 경로 지정하는 방법

html 경우 예

```
<a href="타겟경로">  
  
<script src="타겟경로">  
<link href="타겟경로">  
<form action="타겟경로">
```

js 경우 예

```
location.href="타겟경로"
```

5. 경로(path)

Servlet6.0 및 JSP3.1 Tomcat10

절대경로 (absolute path)

절대적인 기준으로 타겟의 경로가 결정됨.
/ 시작됨. (/컨텍스트명 기준으로 해서 타겟 경로를 찾는다.)

예

현재경로: `http://localhost:8090/01_servlet2/hello.html`

타겟경로: `http://localhost:8090/01_servlet2/world.html`

`<a href="/01_servlet2/world.html">world.html</a>`

타겟경로: `http://localhost:8090/01_servlet2/aaa/happy.html`

`<a href="/01_servlet2/aaa/happy.html">happy.html</a>`

5. 경로(path)

Servlet6.0 및 JSP3.1 Tomcat10

상대경로 (relative path)

기준은 현재 컴포넌트의 url 기준

.(현재 디렉토리)

..(부모 디렉토리)

예

현재경로: `http://localhost:8090/01_servlet2/hello.html`

타겟경로: `http://localhost:8090/01_servlet2/world.html`

`<a href="world.html">world.html</a>`

타겟경로: `http://localhost:8090/01_servlet2/aaa/happy.html`

`<a href="aaa/happy.html">happy.html</a>`

4장. 서블릿 핵심 API

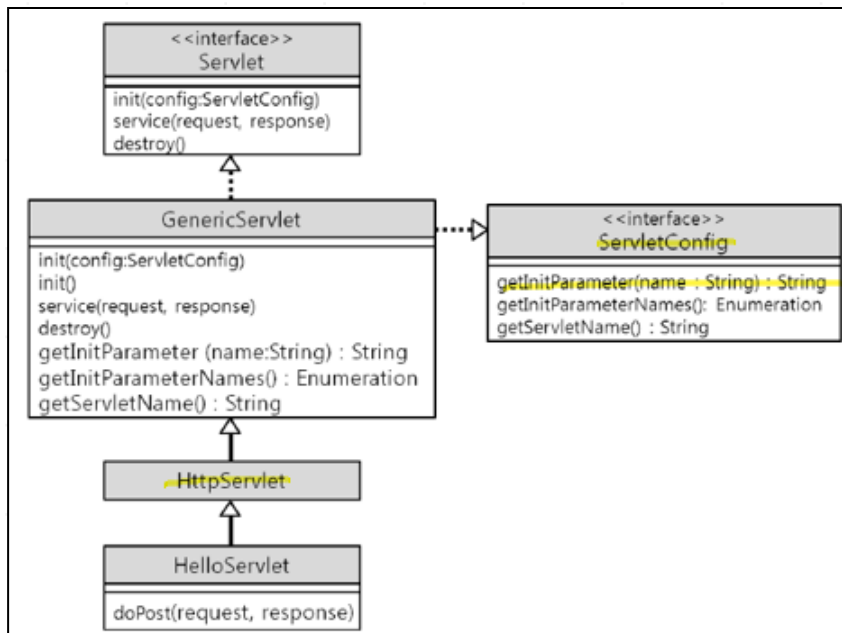
1. ServletConfig API
2. ServletContext API
3. Filter API
4. url-pattern

개요

서블릿이 초기화될 때 외부파일 경로 및 DB 정보 등과 같은 값을 사용해야 되는 경우가 있다. 이때 이런 정보들을 서블릿에서 직접 설정하지 않고 web.xml 파일에 설정한 후에 서블릿에서 참조하여 사용할 수 있다.

이렇게 web.xml 에 설정된 설정값을 초기화 파라미터(initialization parameter)라고 부르고 ServletConfig API를 이용하여 접근할 수 있다.

주의할 점은 서블릿당 하나의 ServletConfig 가 생성되기 때문에 여러 서블릿에서 공유해서 사용하지 못하고 <init-param> 태그로 등록된 서블릿에서만 사용이 가능하다.



계층구조에서 보이는 것처럼 HttpServlet 하나당 하나의 ServletConfig 객체가 생성된다.
따라서 사용자가 만든 서블릿에서만 초기화 파라미터 값을 사용할 수 있다.

web.xml 설정

```
<servlet>
  <servlet-name>서블릿별명</servlet-name>
  <servlet-class>패키지를 포함한 서블릿명</servlet-class>
  <init-param>
    <param-name>초기화파라미터 이름</param-name>
    <param-value>초기화파라미터 값</param-value>
  </init-param>
  <init-param>
    <param-name>초기화파라미터 이름</param-name>
    <param-value>초기화파라미터 값</param-value>
  </init-param>
</servlet>
<servlet-mapping>
  <servlet-name>서블릿별명</servlet-name>
  <url-pattern>/맵핑명</url-pattern>
</servlet-mapping>
```

[예]

```
<servlet>
  <servlet-name>helloServlet</servlet-name>
  <servlet-class>com.test.HelloServlet</servlet-class>
  <init-param>
    <param-name>dirPath</param-name>
    <param-value>c:\wwwtest</param-value>
  </init-param>
</servlet>
<servlet-mapping>
  <servlet-name>helloServlet</servlet-name>
  <url-pattern>/world</url-pattern>
</servlet-mapping>
```

메서드

리턴 타입	메서드명	내용
String	getInitParameter(name)	name에 해당되는 초기화 파라미터 값을 리턴한다. 만약 지정된 name의 파라미터 값이 없으면 null을 리턴한다.
Enumeration	getInitParameterNames()	모든 초기화 파라미터 name 값을 Enumeration 타입으로 리턴한다. 얻은 name 값을 이용하여 초기화 파라미터 값을 얻는다.
String	getServletName()	요청한 서블릿의 이름을 리턴한다.

실습

```
<servlet>
  <servlet-name>InitParam</servlet-name>
  <servlet-class>com.controller.InitParamServlet</servlet-class>
  <init-param>
    <param-name>dirPath</param-name>
    <param-value>c:\\test</param-value>
  </init-param>
  <init-param>
    <param-name>userid</param-name>
    <param-value>admin</param-value>
  </init-param>
</servlet>
<servlet-mapping>
  <servlet-name>InitParam</servlet-name>
  <url-pattern>/InitParam</url-pattern>
</servlet-mapping>
```

InitParamServlet에
서만 사용 가능

```
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    doPost(request, response);
}

protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    String dirPath = getInitParameter("dirPath");
    String userid = getInitParameter("userid");

    response.setContentType("text/html; charset=utf-8");
    PrintWriter out = response.getWriter();
    out.print("<html><body>");
    out.print("디렉토리 경로 : " + dirPath + "<br>");
    out.print("아이디 값 : " + userid + "<br>");
    out.print("</body></html>");
}
```

개요

ServletContext는 웹 어플리케이션(Context) 당 하나씩만 생성되는 객체로서 다수의 Servlet과 JSP에서 공유해서 사용할 수 있다. (여러 서블릿에서 접근 가능)
또한 ServletContext는 웹 어플리케이션의 lifecycle과 일치하기 때문에 웹 어플리케이션이 Tomcat 컨테이너에 존재한다면 ServletContext를 지속적으로 사용할 수 있다.
(application scope 라고 한다.)

핵심 기능

서블릿에서 물리적인 파일을 접근할 수 있다. (읽기 모드)
application scope에 특정 값을 저장하고 사용할 수 있다.

ServletContext 참조방법

ServletContext 객체를 얻기 위해서는 서블릿의 doGet 메서드에서 `getServletContext()` 메서드를 사용한다.

예 > `ServletContext ctx = getServletContext();`



메서드

리턴 타입	메서드명	내용
String	<code>getInitParameter(name)</code>	<code>name</code> 에 해당되는 컨텍스트 파라미터 값을 리턴한다. 만약 지정된 <code>name</code> 의 파라미터 값이 없으면 <code>null</code> 을 리턴한다.
InputStream	<code>getResourceAsStream(path)</code>	웹 어플리케이션의 <code>path</code> 경로에 해당되는 파일을 읽기모드로 접근가능하다.
void	<code>setAttribute(name,value)</code>	<code>application scope</code> 해당되는 속성 값을 저장할 때 사용한다. 브라우저를 종료해도 속성 값을 사용할 수 있다.
Object	<code>getAttribute(name)</code>	<code>name</code> 에 해당되는 속성 값을 리턴한다.

실습1

```
web.xml
1 <?xml version="1.0" encoding="UTF-8"?>
2 <web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns
3
4 <context-param>
5   <param-name>driver</param-name>
6   <param-value>oracle.jdbc.driver.OracleDriver</param-value>
7 </context-param>
8 <context-param>
9   <param-name>savePath</param-name>
10  <param-value>c:\\save</param-value>
11 </context-param>
12
13 <servlet>
14   <servlet-name>InitParam</servlet-name>
15   <servlet-class>com.controller.InitParam</servlet-class>
16   <init-param>
17     <param-name>dirPath</param-name>
18     <param-value>c:\\test</param-value>
19   </init-param>
```

모든 Servlet에서
사용 가능

```
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    String driver = getServletContext().getInitParameter("driver");
    String savePath = getServletContext().getInitParameter("savePath");

    response.setContentType("text/html; charset=utf-8");
    PrintWriter out = response.getWriter();
    out.print("<html><body>");
    out.print("드라이버명: " + driver + "<br>");
    out.print("저장 경로: " + savePath + "<br>");
    out.print("</body></html>");
}
```


실습2

웹 어플리케이션에서 지속적으로 사용해야 되는 데이터를 ServletContext의 `setAttribute(name, value)`로 저장하고 `getAttribute(name)`로 참조하는 예제.

```
@WebServlet("/ContextSet")
public class ContextSetServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // 속성값 설정
        String name = "홍길동";
        int age = 20;

        getServletContext().setAttribute("name", name);
        getServletContext().setAttribute("age", age);
    }
}
```

```
@WebServlet("/ContextGet")
public class ContextGetServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String name = (String) getServletContext().getAttribute("name");
        int age = (Integer) getServletContext().getAttribute("age");

        response.setContentType("text/html; charset=utf-8");
        PrintWriter out = response.getWriter();
        out.print("<html><body>");
        out.print("이름 : " + name + "<br>");
        out.print("나이 : " + age + "<br>");
        out.print("</body></html>");
    }
}
```

개요

임의의 데이터를 특정 객체에 저장할 수 있다.
객체가 3가지 형태가 있고 lifecycle이 모두 다르다.(중요)

종류

가. HttpServletRequest (**request scope** 라고 부른다.)

- 용도: 요청 ~ 응답 범위에 사용하는 데이터 필요시
- 저장방법: `request.setAttribute(String key, Object value);`
- 조회: `request.getAttribute(key);` 형변환 필요
- 삭제: `request.removeAttribute(key);`

종류

나. HttpSession (session scope 라고 부른다.)

- 용도: 브라우저가 종료되기 전까지 계속적으로 사용하는 데이터 필요시

`HttpSession session = request.getSession();`

- 저장방법: `session.setAttribute(String key, Object value);`
- 조회: `session.getAttribute(key);` 형변환 필요
- 삭제: `session.removeAttribute(key);`

다. ServletContext (application scope 라고 부른다.)

- 용도: Web Application이 종료되기 전까지 계속적으로 사용하는 데이터 필요시

`ServletContext ctx = getServletContext();`

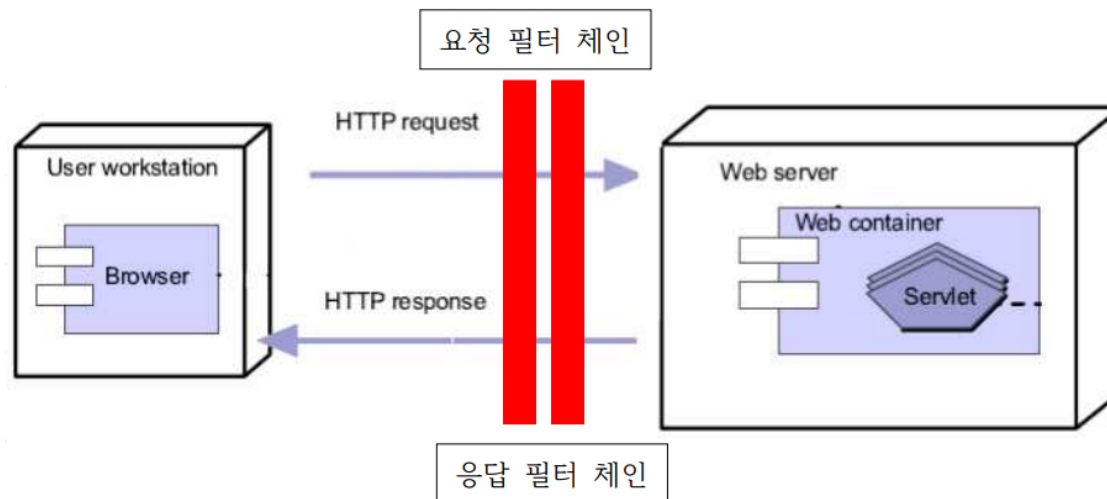
- 저장방법: `ctx.setAttribute(String key, Object value);`
- 조회: `ctx.getAttribute(key);` 형변환 필요
- 삭제: `ctx.removeAttribute(key);`

개요

클라이언트에서 서버로 요청할 때 서블릿이 요청을 받기 전과 응답하기 전에 특정 작업을 수행할 수 있도록 Filter API를 지원한다.

즉 웹 컴포넌트가 실행되기 전의 선처리(pre-processing)작업과 응답되기 전의 후처리(post-processing)작업을 수행하는 API이다.

다수의 Filter를 체인(chain)처럼 묶어서 사용할 수도 있다.



request 요청이 요청 필터체인을 거쳐서 서블릿으로 전송된다. 이때 서블릿이 처리하기 전에 실행해야 되는 선처리 작업을 구현할 수 있다.

response 응답이 응답 필터체인을 거쳐서 전송된다. 요청과 마찬가지로 응답 되기전에 실행 되어야 하는 후처리 작업을 구현할 수 있다.

구현방법

- 1) package 지정 필수
- 2) implements Filter 및 메서드 재정의
- 3) doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
- 4) 필터 매핑 (web.xml 에 설정)

실습

```
public class EncodingFilter implements Filter {  
  
    public void init(FilterConfig fConfig) throws ServletException {  
        System.out.println("EncodingFilter 초기화");  
    }  
  
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)  
        throws IOException, ServletException {  
        System.out.println("EncodingFilter 요청필터 코드작업");  
        request.setCharacterEncoding("utf-8");  
        chain.doFilter(request, response);  
        System.out.println("EncodingFilter 응답필터 코드작업");  
    }  
  
    public void destroy() {  
        System.out.println("EncodingFilter 제거");  
    }  
}
```

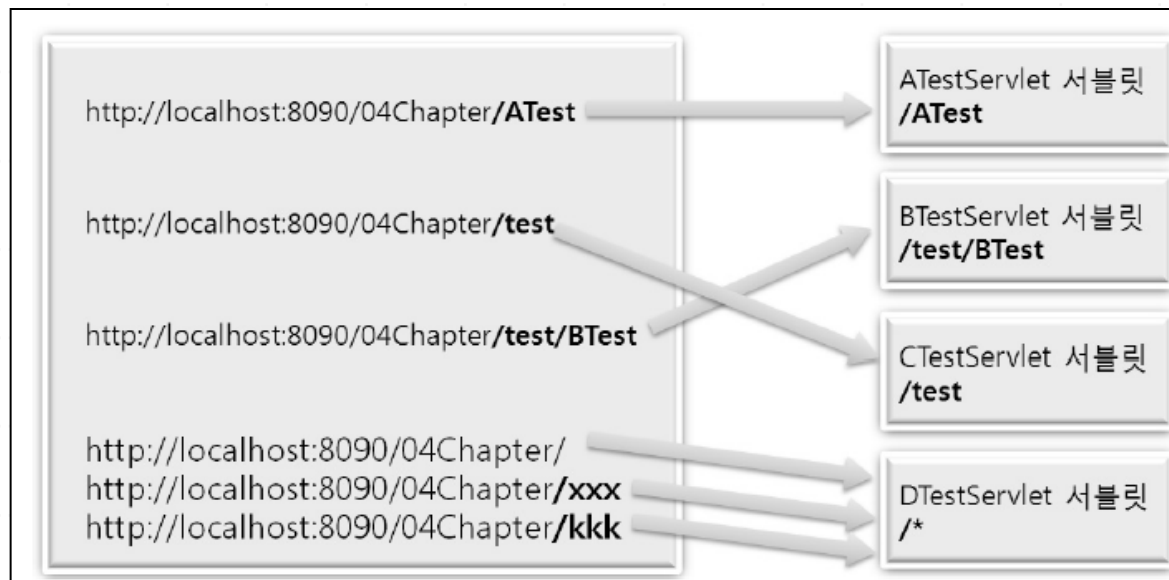
```
<!-- Filter 등록 -->  
<filter>  
    <filter-name>myFilter</filter-name>  
    <filter-class>com.controller.EncodingFilter</filter-class>  
</filter>  
<filter-mapping>  
    <filter-name>myFilter</filter-name>  
    <url-pattern>*/</url-pattern>  
</filter-mapping>
```

개요

url-pattern은 클라이언트에서 요청하는 URL 값의 패턴에 따라서 서버에 저장된 어떤 웹 컴포넌트가 실행될지를 결정하는 방법이다.

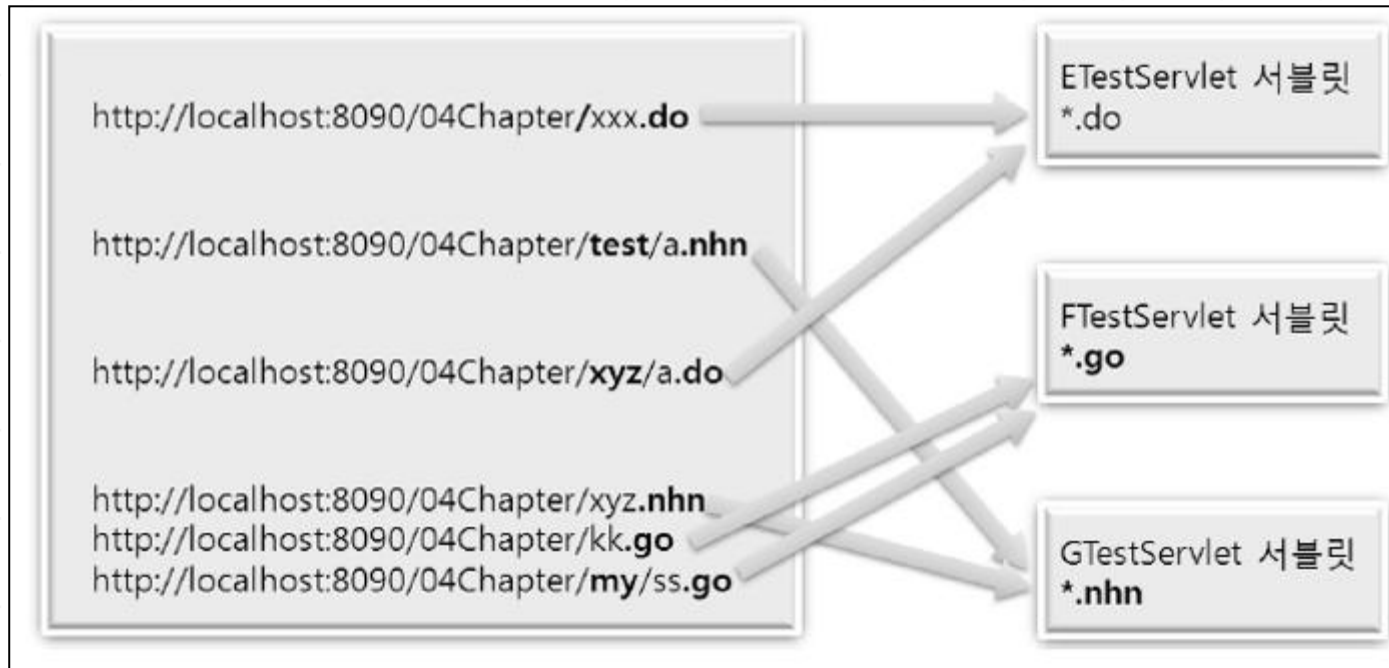
1) 디렉터리 패턴

/디렉터리 형식으로 지정해서 사용되는 방법으로서 sub구조를 가질 수 있다.
만약 **/*** 로 지정하면 클라이언트가 요청하는 URL값의 패턴과 무관하게 항상 수행된다.



2) 확장자 패턴

*.확장자 형식으로 지정해서 사용하는 방식으로 / 로 시작하지 않는다.

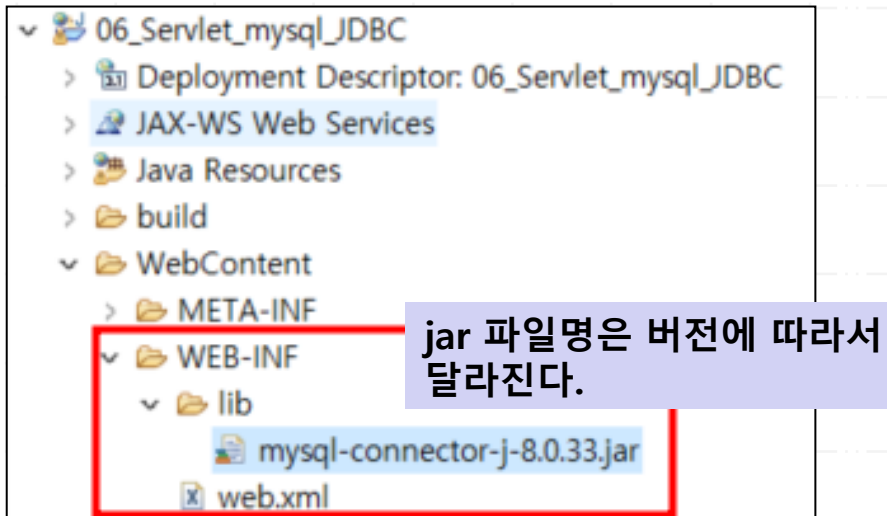


5장. DB 연동

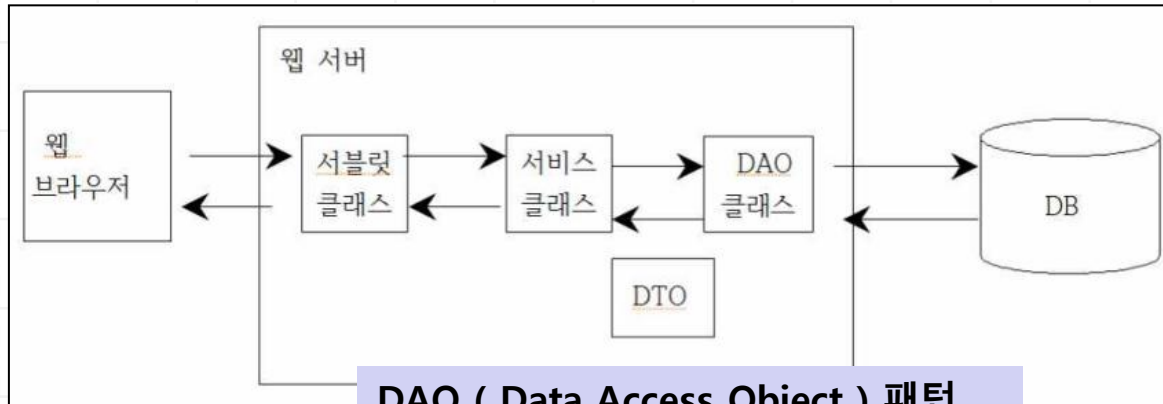
1. JDBC 환경설정
2. 웹 어플리케이션 구조
3. DAO 및 DTO 패턴
4. 트랜잭션 처리

환경설정

이클립스에 MySQL에서 제공하는 드라이버(*.jar)를 설정하기 위해서는 웹 프로젝트내의 **WEB-INF/lib** 폴더에 드라이버를 복사하면 된다.



개요



DAO (Data Access Object) 패턴
DTO (Data Transfer Object) 패턴

DTO 예

```
package com.dto;

public class EmpDTO {

    private int empno;
    private String ename;
    private int sal;
    private int deptno;

    public EmpDTO() {}

    public EmpDTO(int empno, String ename, int sal, int deptno) {
        this.empno = empno;
        this.ename = ename;
        this.sal = sal;
        this.deptno = deptno;
    }
}
```

DAO 예

```
public class EmpDAO {  
  
    public List<EmpDTO> select(Connection con){  
        List<EmpDTO> list = new ArrayList<EmpDTO>();  
        PreparedStatement pstmt = null;  
        ResultSet rs = null;  
        try{  
            String sql = "select empno, ename, sal, deptno from emp";  
            pstmt = con.prepareStatement(sql);  
            rs = pstmt.executeQuery();  
            while(rs.next()){  
                int empno = rs.getInt("empno");  
                String ename = rs.getString("ename");  
                int sal = rs.getInt("sal");  
                int deptno = rs.getInt("deptno");  
                EmpDTO dto = new EmpDTO(empno, ename, sal, deptno);  
                list.add(dto);  
            }  
        }catch(Exception e){  
            e.printStackTrace();  
        }finally{  
            try {  
                if(rs!=null) rs.close();  
                if(pstmt!=null) pstmt.close();  
            } catch (SQLException e) {  
                e.printStackTrace();  
            }  
        }  
        return list;  
    }  
}
```

개요

JDBC에서는 기본적으로 모든 DML 작업이 자동으로 commit 된다. (auto commit)
하지만 트랜잭션(transaction)으로 처리해야 하는 작업에서는 명시적으로 auto
Commit 기능을 비활성으로 처리한 후에 작업해야 된다.
Connection의 setAutoCommit(false) 메서드를 사용하여 비활성하고 commit() 또는
rollback() 메서드를 사용하여 트랜잭션을 종료할 수 있다.

```
@Override
public int transactionMethod(DeptDTO dto) throws RecordNotFoundException {
    Connection con = null;
    int n = 0;
    try {
        con = DriverManager.getConnection(url, userid, passwd);
        // autocommit 비활성화
        con.setAutoCommit(false);

        DeptDAO dao = new DeptDAO();
        n = dao.update(con, dto);
        n = dao.delete(con, 10 );

        con.commit();
    } catch (SQLException e) {
        try {
            if(con!=null)con.rollback();
        } catch (SQLException e1) {
            e1.printStackTrace();
        }
        e.printStackTrace();
    } finally {
        try {
            if(con!=null)con.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
    return n;
} //end transactionMethod
```

6장. 서블릿 심화

1. FrontController 패턴
2. 요청 위임 (forward, redirect)
3. 세션 관리 (session tracking)
4. 파일 업로드 (Apache Commons Fileupload)

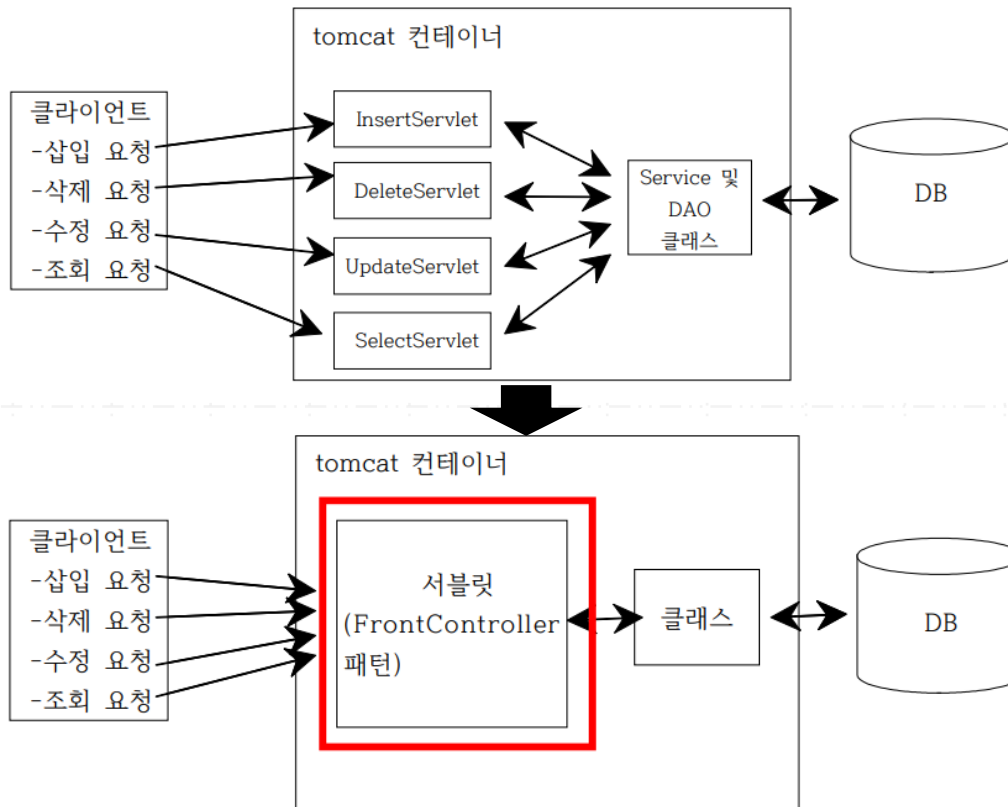
1. FrontController 패턴

Servlet6.0 및 JSP3.1 Tomcat10

개요

웹 어플리케이션 개발시 사용자의 다양한 요청을 효과적으로 처리하기 위해서 최초 진입점(initial point)을 정의하고 사용하는 패턴.

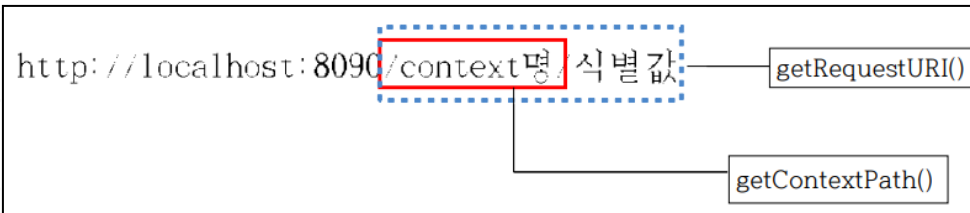
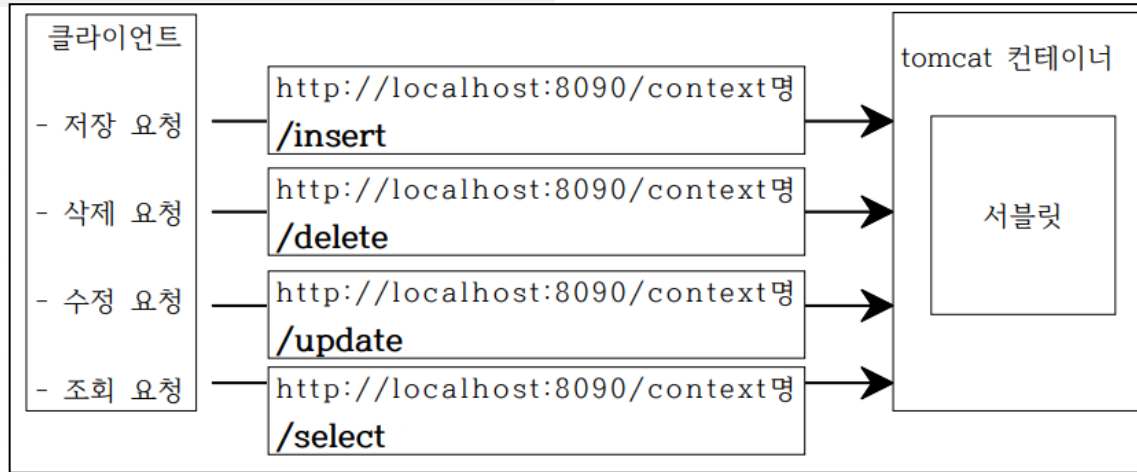
모든 사용자의 요청을 집중화 시키면 요청을 분산시켜 발생하는 중복된 코드를 제거할 수 있으며 또한 사용자의 요청을 일관된 방법으로 관리할 수 있다.



1. FrontController 패턴

Servlet6.0 및 JSP3.1 Tomcat10

구현



```
String requestURI = request.getRequestURI();  
String contextPath = request.getContextPath();  
String command = requestURI.substring(contextPath.length());
```

```
if( command.equals( "/insert" ) ){  
    //저장  
}else if( command.equals( "/delete" ) ){  
    //삭제  
}
```

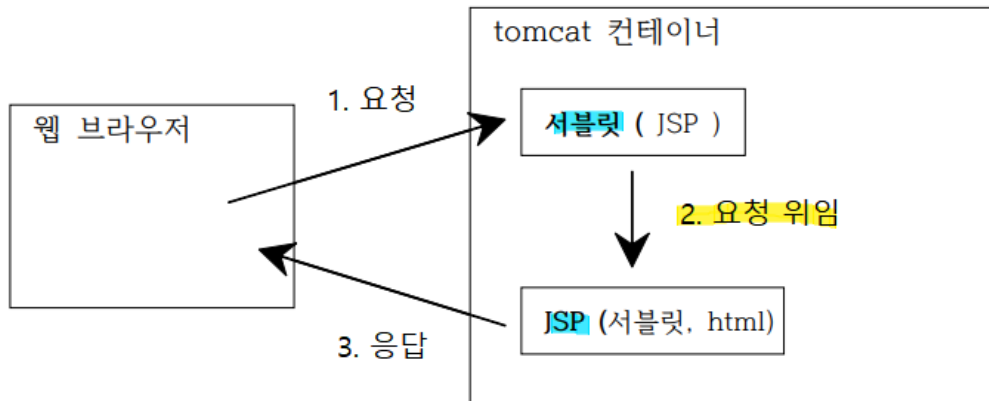
2. 요청 위임

Servlet6.0 및 JSP3.1 Tomcat10

개요

클라이언트의 요청을 받은 서블릿에서 다른 컴포넌트(서블릿, **JSP**, html)로 요청을 위임할 수 있는 방법이다.

직접 요청 받은 서블릿에서 모든 작업을 처리하지 않고 위임한 컴포넌트에서 특정 작업을 처리하여 모듈화가 가능하다.



위임 방법 2가지

HttpServletRequest 객체를 이용한 forward 방법.

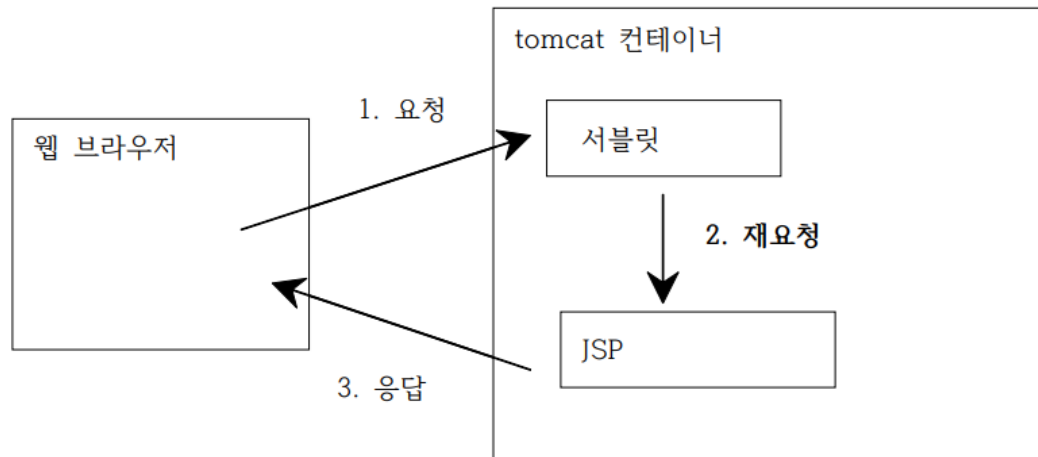
HttpServletResponse 객체를 이용한 redirect 방법.

2. 요청 위임

Servlet6.0 및 JSP3.1 Tomcat10

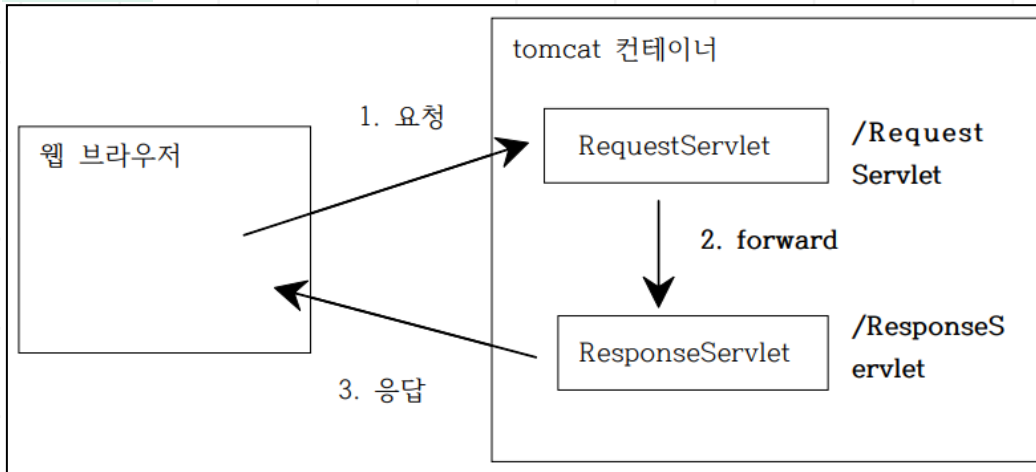
1) 포워드 방법

맨 처음 요청 받은 서블릿에서 생성된 `HttpServletRequest` 객체를 재사용하여 위임하는 방법으로 포워드(forward)라고 부른다.
하나의 `HttpServletRequest` 를 사용하기 때문에 forward 되었을 때 URL이 변경이 안된다.



```
RequestDispatcher dis = request.getRequestDispatcher(target);  
dis.forward( request, response );
```

실습



```
@WebServlet("/RequestServlet")
public class RequestServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        //속성 설정
        request.setAttribute("username", "홍길동");
        request.setAttribute("useraddress", "서울");
        //forward
        RequestDispatcher dis = request.getRequestDispatcher("ResponseServlet");
        dis.forward(request, response);
    }
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        doGet(request, response);
    }
}
```

2. 요청 위임

Servlet6.0 및 JSP3.1 Tomcat10

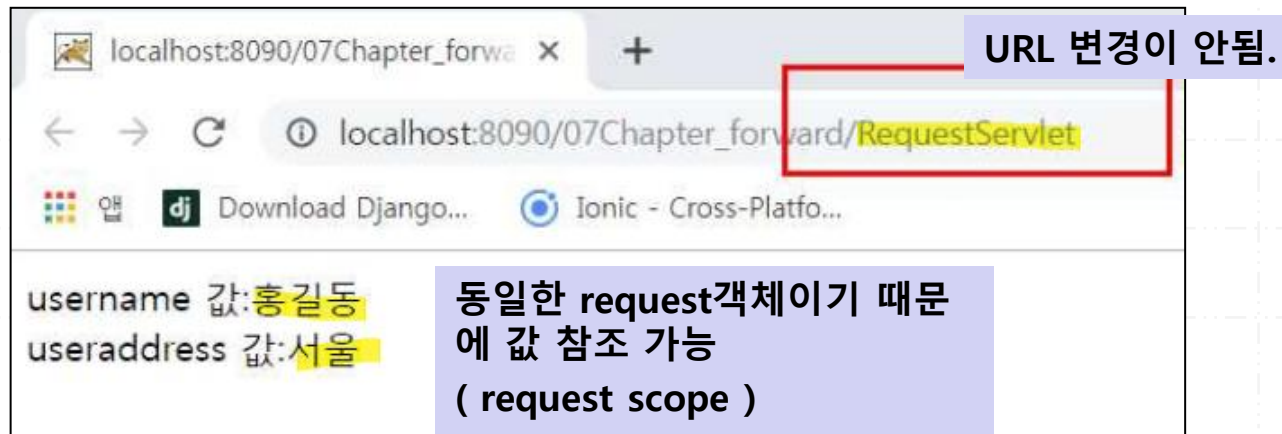
```
@WebServlet("/ResponseServlet")
public class ResponseServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String username = (String) request.getAttribute("username");
        String useraddress = (String) request.getAttribute("useraddress");

        response.setContentType("text/html; charset=utf-8");
        PrintWriter out = response.getWriter();
        out.print("<html><body>");
        out.print("username 값:" + username + "<br>");
        out.print("useraddress 값:" + useraddress + "<br>");
        out.print("</body></html>");
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        doGet(request, response);
    }
}
```



URL 변경이 안됨.

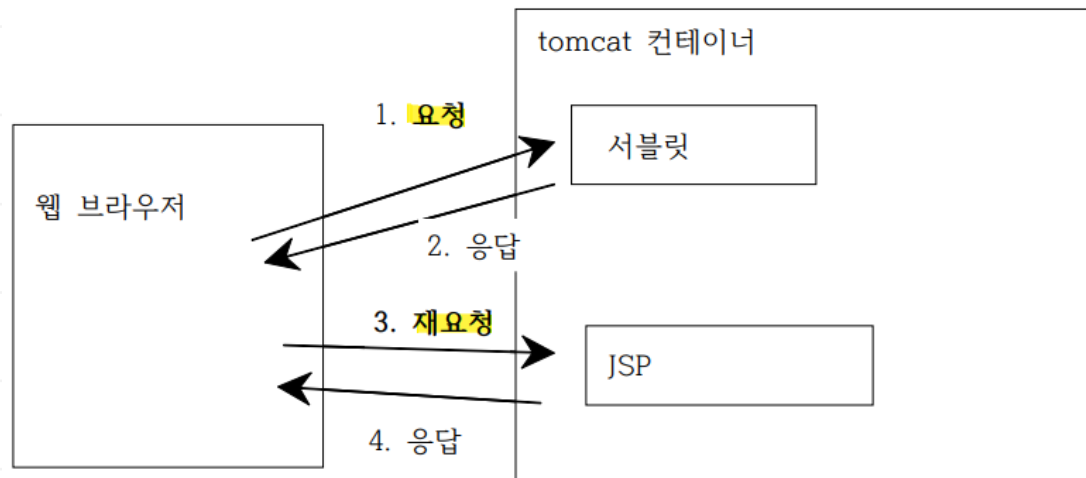
username 값:홍길동
useraddress 값:서울

동일한 request 객체이기 때문에 값 참조 가능
(request scope)

2) 리다이렉트 방법

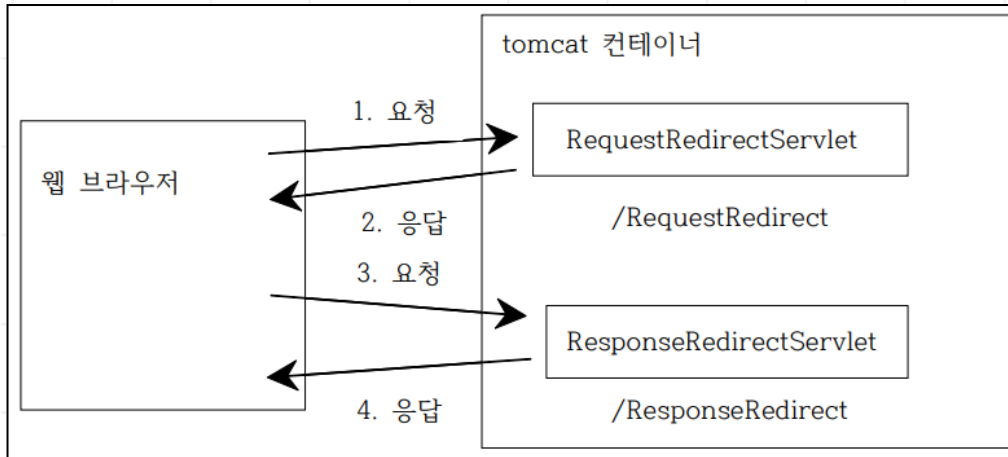
포워드 방법과는 다르게 맨 처음 요청 받은 서블릿에서 생성된 `HttpServletRequest` 객체가 아닌 새로운 `HttpServletRequest` 객체를 생성하여 위임하는 방법으로 리다이렉트(`redirect`)라고 부른다.

서로 다른 `HttpServletRequest` 객체를 사용하기 때문에 `redirect` 되었을 때 URL이 변경된다.



```
response.sendRedirect(target);
```

실습



```
@WebServlet("/RequestRedirect")
public class RequestRedirectServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        //속성 설정
        request.setAttribute("username", "홍길동");
        request.setAttribute("useraddress", "서울");

        //redirect
        response.sendRedirect("ResponseRedirect");
    }
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        doGet(request, response);
    }
}
```

2. 요청 위임

Servlet6.0 및 JSP3.1 Tomcat10

```
@WebServlet("/ResponseRedirect")
public class ResponseRedirectServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String username = (String) request.getAttribute("username");
        String useraddress = (String) request.getAttribute("useraddress");

        response.setContentType("text/html; charset=utf-8");
        PrintWriter out = response.getWriter();
        out.print("<html><body>");
        out.print("username 값:" + username + "<br>");
        out.print("useraddress 값:" + useraddress + "<br>");
        out.print("</body></html>");
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        doGet(request, response);
    }
}
```

localhost:8090/07Chapter_redire x + URL 변경이 됨.

localhost:8090/07Chapter_redirec/ResponseRedirect

username 값:null
useraddress 값:null

다른 request객체이기 때문에
값 참조 불가능

개요

일반적으로 사용되는 세션의 정의는 서버와 클라이언트 간의 지속적인 연결을 의미한다. 연결을 통하여 클라이언트는 지속적으로 서버에 특정 동작을 요청할 수 있으며 서버는 실행 결과를 클라이언트에 응답할 수 있다.

http 기반의 세션은 불특정 다수인 클라이언트와 지속적인 연결방식으로는 웹 서버의 부하가 매우 크기 때문에 요청하고 응답 받으면 즉시 연결을 끊는 방식으로 동작이 된다.

(connectionless, stateless)

이런 특징은 웹 페이지 간에 연결 고리가 없다는 것을 의미한다. 즉 첫 화면에서 선택한 물건을 장바구니에 담고 다음 페이지에서 결제할 때 이전 화면의 장바구니에 담긴 정보를 확인할 수 없다는 것이다.

이처럼 http 기반의 특징을 극복하기 위해서 사용자의 상태 정보를 관리하는 메커니즘이 필요한데 이것을 세션관리(session tracking)라고 부른다.

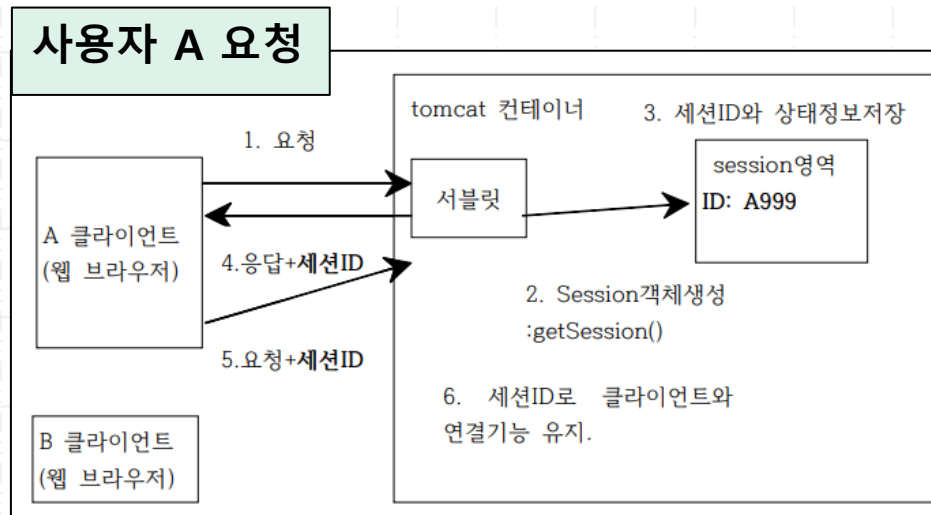
세션 방법 2가지

HttpSession API를 이용한 세션 처리 방법.
Cookie API를 이용한 쿠키 처리 방법.

개요

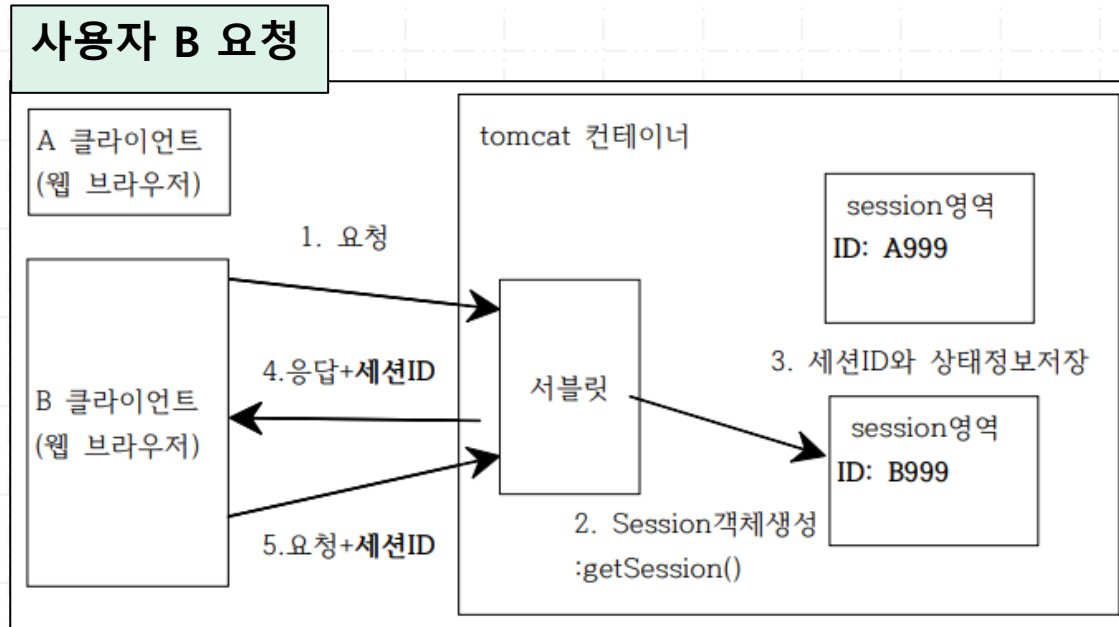
세션(session)은 사용자의 상태 정보를 서버에서 관리하는 메커니즘을 의미한다.
세션의 정보는 기본적으로 클라이언트가 서버에 접속해서 브라우저를 종료할 때까지 유지된다.(session scope)
상태정보가 서버에 저장되기 때문에 서버의 부하가 클 수 있으나 보안에 덜 취약하다.
따라서 time-out 제한을 두어 일정시간(기본 30분)동안 새로운 요청이 없으면 세션 정보를 유지하지 않고 제거한다.

아키텍처



4. HttpSession

Servlet6.0 및 JSP3.1 Tomcat10



메서드

메서드명	설 명
request.getSession()	session 영역을 얻을 때 사용하며 session 영역이 없으면 새로 생성하고, 있으면 생성된 영역을 참조한다. request.getSession(true) 메서드와 동일한 기능이다. 일반적으로 세션을 처음 생성하는 서블릿에서 지정한다.
request.getSession(false)	session 영역을 얻을 때 사용하며 session 영역이 없으면 null 값을 리턴하고, 있으면 생성된 영역을 참조한다. 일반적으로 세션을 사용하는 서블릿에서 주로 사용된다.
sess.getId()	생성된 세션 ID 값을 반환한다.
sess.setAttribute(key,value)	session 영역에 속성(Attribute) 값을 설정한다. session scope를 따르기 때문에 브라우저를 종료하기 전까지 사용 가능하다. 단, 기본으로 30분 이상 요청이 없을 시 제거된다.
sess.getAttribute(key)	session 영역에 저장된 속성 값을 반환한다.
sess.removeAttribute(key)	session 영역에 저장된 속성을 제거한다.
setMaxInactiveInterval(sec)	지정된 시간만큼 세션을 유지시킨다. 단위는 초(second).
getMaxInactiveInterval()	세션의 유지시간을 반환한다.
sess.invalidate()	즉시 session 영역을 제거한다. 로그아웃 기능 시 사용된다.

세션 얻는 방법1

session 영역이 있으면 반환하고 없으면 새로 생성되어 반환한다.

예>

```
HttpSession session = request.getSession();  
HttpSession session = request.getSession(true);
```

세션 얻는 방법2

session 영역이 있으면 반환하고 없으면 null 반환한다.

예>

```
HttpSession session = request.getSession(false);
```

세션에 상태정보 저장 및 조회, 제거

session 영역에 필요한 값을 저장하고 조회하거나 삭제할 수 있다.
이때 세션에 저장된 상태정보는 session scope 를 따르기 때문에 브라우저의 lifecycle과 동일하다.

예>

```
HttpSession session = request.getSession();
```

```
session.setAttribute(String key, Object obj);
```

```
session.getAttribute(String key); // 형변환 필요
```

```
session.removeAttribute(String key);
```

세션영역 제거

1) time-out 지정하여 제거하는 방법

- `setMaxInactiveInterval(Millisecond)` 메소드 사용

```
session.setMaxInactiveInterval( 60*60*24 ); //24시간 유지
```

- `web.xml` 사용

```
<web-app>
  <!-- 단위는 분(minute) -->
  <session-config>
    <session-timeout>60</session-timeout>
  </session-config>
</web-app>
```

2) 즉시 제거하는 방법

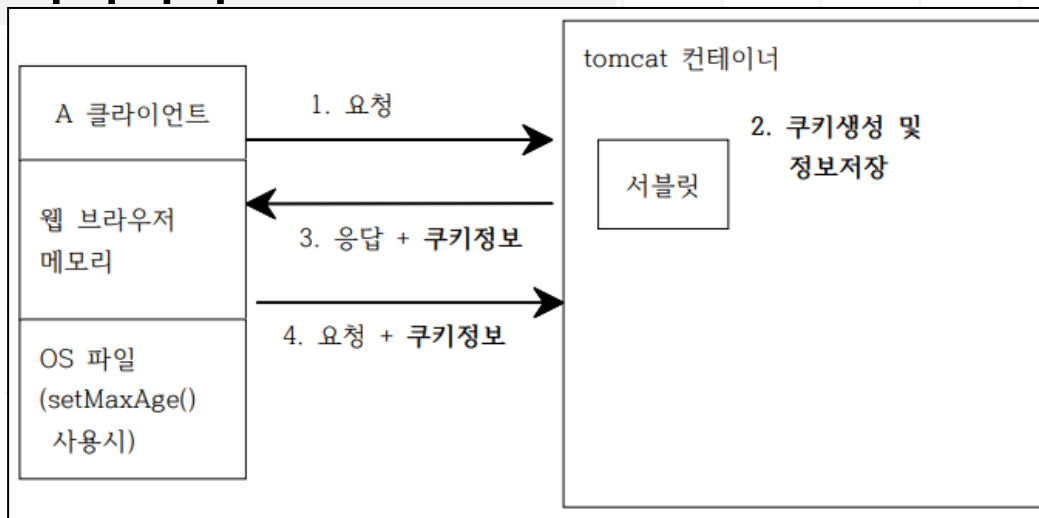
- `invalidate()` 메소드 사용

```
session.invalidate(); // 즉시 제거
```

개요

쿠키는 사용자의 상태정보를 클라이언트에서 관리하는 메커니즘을 의미한다.
클라이언트에 정보가 저장되기 때문에 서버 부하가 크지 않으나 누구나 클라이언트에 접근할 수 있기 때문에 보안에 매우 취약하다.
세션과 다르게 문자열만 저장이 가능하고 domain 당 300개의 쿠키정보만 저장할 수 있다.
가장 큰 단점은 클라이언트에서 쿠키 기능을 비활성화 시킬 수 있기 때문에 사용에 제약이 있다.
쿠키 정보는 브라우저의 메모리 또는 OS 파일로 저장할 수 있다. 기본 저장은 브라우저 메모리 이기 때문에 브라우저를 종료하면 자동으로 쿠키 정보도 제거된다.
세션과 동일하게 time-out 지정도 가능하다.

아키텍처



메서드

생성자 및 메서드명	설 명
Cookie(name, value)	쿠키 생성시 사용되는 생성자이다.
response.addCookie(cookie)	생성된 쿠키를 응답처리 한다.
request.getCookies()	클라이언트로부터 쿠키 정보를 배열로 리턴받는다.
setMaxAge(second)	쿠키 만료시간을 설정한다.
getMaxAge()	쿠키 만료시간을 얻는다.
getName()	쿠키 이름을 얻는다.
setValue(value)	쿠키 값을 설정한다.
getValue()	쿠키 값을 얻는다.
length	쿠키 개수를 리턴한다.

쿠키 생성

사용자 정보를 저장하기 위해서 Cookie 클래스를 객체 생성한다.

예>

```
Cookie c = new Cookie(String key, String value);
```

클라이언트에 전달

사용자 정보를 저장하기 위해서 Cookie 클래스를 객체 생성한다.

예>

```
Cookie c = new Cookie(String key, String value);
```

```
c.setMaxAge(-1);
```

```
response.addCookie( c );
```


쿠키 참조

서버 요청시 전달된 쿠키 정보를 HttpServletRequest 사용하여 참조한다.

예>

```
Cookie [] cookies = request.getCookies();
```

개발자도구 > Application

The screenshot shows the Chrome DevTools Application tab. The left sidebar has a tree view with 'Application' expanded, showing 'Manifest', 'Service workers', and 'Storage'. Under 'Storage', 'Cookies' is selected, and 'http://localhost:8090' is highlighted. The main pane displays a table of cookies.

Name	Value	D.	P.	Expires / Max-...
JSESSIONID	08D774BF0A1902A3E...	I...	/...	Session
username	홍길동	I...	/...	2024-08-16T03:...

개요

파일 업로드 기능을 구현하기 위한 다양한 방법 중에서 현재까지 가장 많이 사용되고 있는 검증된 방법은 Apache 그룹에서 제공하는 Commons Fileupload 라이브러리를 사용하는 것이다. Commons Fileupload 라이브러리는 Spring 프레임워크와 같은 유명한 프레임워크에서도 사용되는 매우 안정적인 라이브러리이다.

파일업로드 문서

<https://commons.apache.org/proper/commons-fileupload/using.html>

Servlets, Jakarta Servlets, and Portlets

Starting with version 1.1, FileUpload supports file upload requests in both servlet and portlet environments. The usage is almost identical in both environments.

If you are building a portlet application, the following are the two distinctions you should make as you read this document:

- Where you see references to the `JakartaServletFileUpload` class, substitute the `JavaxPortletFileUpload` class.
- Where you see references to the `HttpServletRequest` class, substitute the `ActionRequest` class.

Version 2 of FileUpload introduces support for the Jakarta Servlet API 5. (This API is the successor to the classic servlet environment.) When building a Jakarta Servlet application, keep the following in mind, as you read this document:

- Where you see references to the `ServletFileUpload` class, substitute the `JakartaServletFileUpload` class.
- Likewise, references to the `FileCleanerCleanup` class should be substituted with the `JakartaServletFileCleaner` class.
- Where you see references to the `HttpServletRequest` class (as in `javax.servlet.http.HttpServletRequest`), then substitute the `HttpServletRequest` class with a single import statement.

The simplest case

The simplest usage scenario is the following:

- Uploaded items should be retained in memory as long as they are reasonably small.
- Larger items should be written to a temporary file on disk.
- Very large upload requests should not be permitted.
- The built-in defaults for the maximum size of an item to be retained in memory, the maximum permitted size of an upload request, and the maximum number of items to be retained in memory.

Handling a request in this scenario couldn't be much simpler:

```
// Create a factory for disk-based file items
DiskFileItemFactory factory = new DiskFileItemFactory();

// Configure a repository (to ensure a secure temp location is used)
ServletContext servletContext = this.getServletConfig().getServletContext();
File repository = (File) servletContext.getAttribute("jakarta.servlet.context.tempdir"); // Or "java.io.tmpdir"
factory.setRepository(repository);

// Create a new file upload handler
JakartaServletDiskFileUpload upload = new JakartaServletDiskFileUpload(factory);

// Parse the request
List<DiskFileItem> items = upload.parseRequest(request);
```

의존성 설정

```
<!-- https://mvnrepository.com/artifact/org.apache.commons/commons-fileupload2-jakarta -->
<dependency>
  <groupId>org.apache.commons</groupId>
  <artifactId>commons-fileupload2-jakarta</artifactId>
  <version>2.0.0-M1</version>
</dependency>
```

▼ Maven Dependencies

- > commons-fileupload2-jakarta-2.0.0-M1.jar
- > commons-fileupload2-core-2.0.0-M1.jar -
- > commons-io-2.13.0.jar - C:\Users\Winky4\

이클립스 라이브러리 설정

▼ WEB-INF

▼ lib

- commons-fileupload2-core-2.0.0-M1.jar
- commons-fileupload2-jakarta-2.0.0-M1.jar
- commons-io-2.16.1.jar

샘플

```
<form method="POST" enctype="multipart/form-data" action="fup.cgi">
  File to upload: <input type="file" name="upfile"><br/>
  Notes about the file: <input type="text" name="note"><br/>
  <br/>
  <input type="submit" value="Press"> to upload the file!
</form>
```

```
boolean isMultipart = JakartaServletFileUpload.isMultipartContent(request);
if (isMultipart) {
    DiskFileItemFactory factory = DiskFileItemFactory.builder().get();
    JakartaServletDiskFileUpload upload = new JakartaServletDiskFileUpload(factory);

    List<DiskFileItem> items = upload.parseRequest(request);
    String fileName = null;
    for (FileItem item : items) {
        if (item.isFormField()) {
            String name = item.getFieldName();
            String value = item.getString();
        } else {
            fileName = item.getName();
            File f = new File("c:\\www\\upload", fileName);
            item.write(f.toPath());
        }
    }
} else {
    System.out.println("파일요청 아님");
}
```

7장. JSP

1. JSP 개요
2. JSP 태그
3. 내장 객체 (내장 변수)
4. EL 태그
5. JSTL 태그
6. MVC 패턴 (Model 2 Architecture)

개요

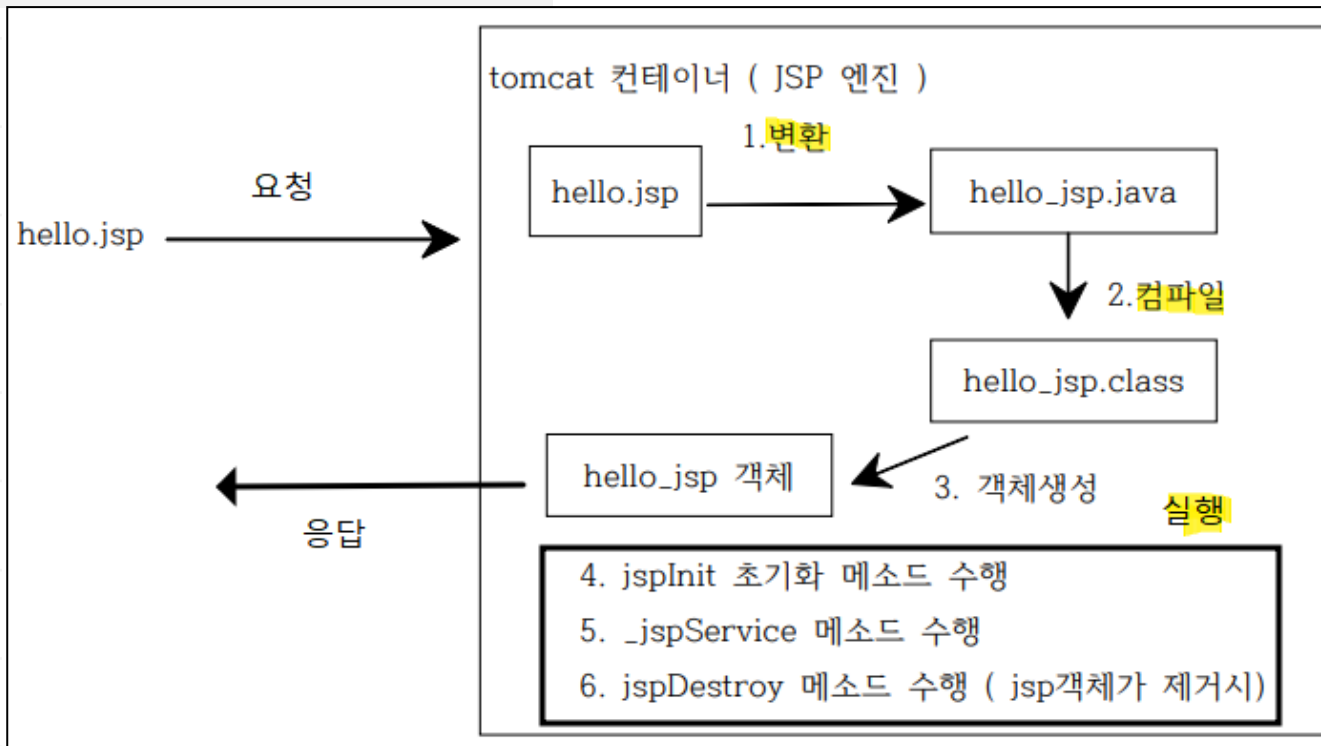
JSP는 서블릿과 동일하게 동적인 웹 어플리케이션을 개발할 때 사용한다. 차이점은 서블릿은 Java 파일이기 때문에 비즈니스 로직 구현에 최적화 되어 있고 JSP는 태그(html 및 JSP, EL, JSTL)로 구성되어 있기 때문에 화면 로직 구현에 최적화 되어있다.

MVC 패턴에서 화면처리 용도인 View역할을 JSP가 담당한다.

서블릿 vs JSP

	서블릿	JSP
형식		
특징	Java 코드 내에 HTML 코드가 삽입되는 형태이다. 따라서 HTML를 작성하는 Java 코드 작업이 복잡하다.	Java 코드 내에 HTML 코드가 삽입되는 형태이다. 서블릿에 비해서 HTML 코드를 쉽게 작성할 수 있다.
목적	Java 코드를 이용한 Business Logic 처리에 적합하다. 따라서 MVC 패턴의 Controller 역할로 사용된다.	tag를 이용한 Presentation Logic 처리에 적합하다. 따라서 MVC 패턴의 View 역할로 사용된다.

아키텍처



Tomcat의 **work** 디렉터리에서 변환된 소스파일과 컴파일 파일 확인 가능.

스크립팅 요소

스크립팅 요소	예 제
Comment tag	<%-- 주식 --%>
Directive tag	<%@ 지시자 %>
Declaration tag	<%! 자바 선언문 %>
Scriptlet tag	<% 자바 코드 %>
Expression tag	<%= 자바 표현식 %>

3. Directive tag 지시어

Servlet6.0 및 JSP3.1 Tomcat10

page 지시어

JSP 페이지에서 설정 가능한 속성들을 지시하는 용도로 사용한다.

```
<%@ page 속성명="속성 값" 속성명2="속성 값2" %>
```

속성명	기본값	설 명	예
contentType	text/html	응답 시의 MIME 타입	contentType="text/html"
import	없음	import할 패키지 지정	import="java.util.ArrayList"
session	true	HttpSession 사용 여부	session="true"
errorPage	없음	에러 발생 시 처리할 에러 페이지 지정	errorPage="error.jsp"
isErrorPage	false	현재 페이지를 에러 페이지로 지정	isErrorPage="false"
language	java	사용할 언어 설정	language="java"
extends	없음	상속받은 클래스명 지정	extends="패키지포함 클래스명"
buffer	8kb	출력 버퍼 크기 지정	buffer="8kb"

3. Directive tag 지시어

Servlet6.0 및 JSP3.1 Tomcat10

include 지시어

현재 페이지에서 다른 html 및 JSP 페이지를 정적(static) 방식으로 삽입할 수 있다.

```
<%@ include file="포함할 페이지" %>
```

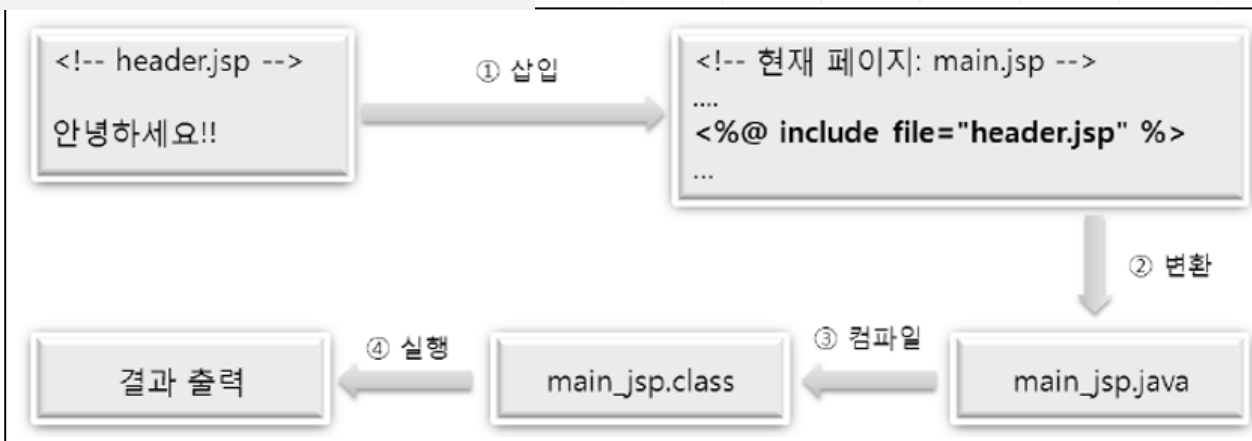
- 정적 include 방법

```
<%@ include file="포함할 페이지" %>
```

- 동적 include 방법 (include 액션 태그)

```
<jsp:include page="포함할 페이지" flush="true" />
```

동작 방식



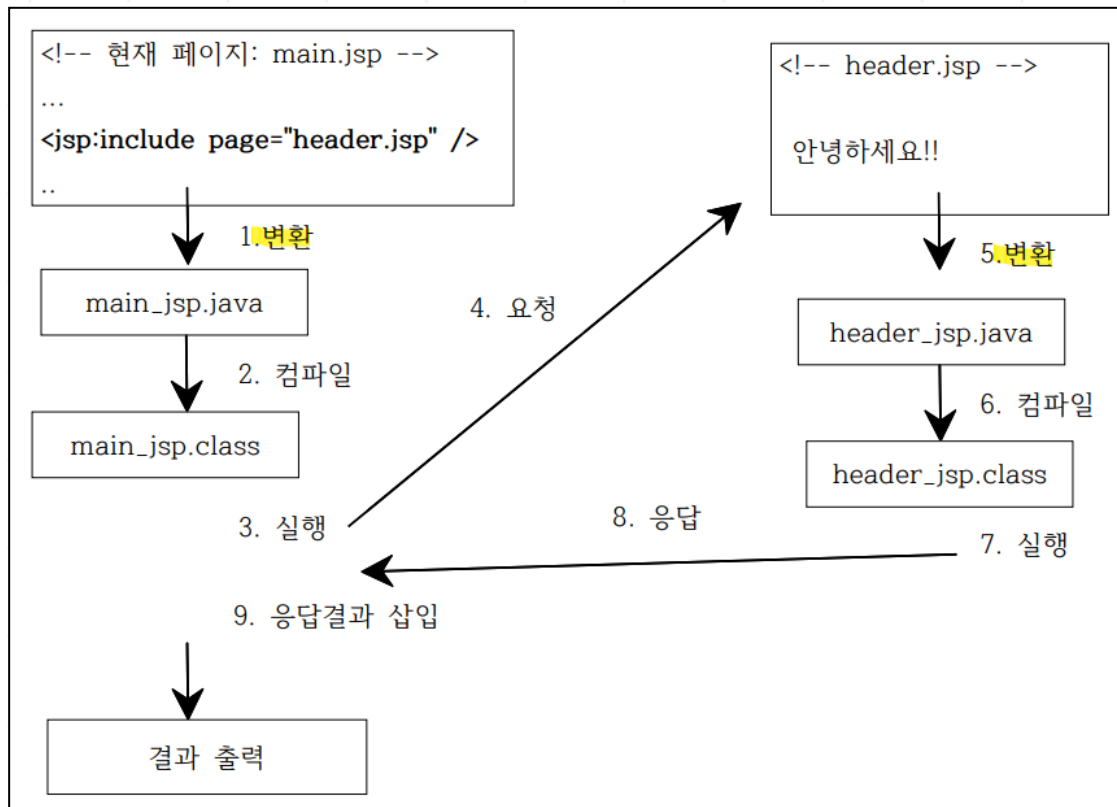
3. Directive tag 지시어

Servlet6.0 및 JSP3.1 Tomcat10

include 액션태그

현재 페이지에서 다른 html 및 JSP 페이지를 동적(dynamic) 방식으로 삽입할 수 있다.

동작 방식



3. Directive tag 지시어

Servlet6.0 및 JSP3.1 Tomcat10

taglib 지시어

JSP에서 외부 라이브러리로 만든 태그(tag)를 지정할 때 사용한다.
대표적으로 JSTL(JSP Standard Tag Library) 이다.

```
<%@ taglib uri="TLD파일 URI" prefix="네임스페이스명" %>
```

JSTL 라이브러리 사용

라이브러리	URI	Prefix	예 제
Core	http://java.sun.com/jsp/jstl/core	c	<c:tag ...>
XML Processing	http://java.sun.com/jsp/jstl/xml	x	<x:tag ..>
l18N formatting	http://java.sun.com/jsp/jstl/fmt	fmt	<fmt:tag ..>
SQL	http://java.sun.com/jsp/jstl/sql	sql	<sql:tag ..>
Functions	http://java.sun.com/jsp/jstl/functions	fn	fn:function(...)

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

4. Declaration tag 지시어

Servlet6.0 및 JSP3.1 Tomcat10

개요

JSP 페이지에서 자바의 멤버(변수와 메서드)를 선언할 때 사용된다.

```
<%!  
    멤버 변수 선언;  
    메서드 선언;  
%>
```

샘플

```
<%! int i; %>  
<%! int i=0; %>  
<%! public void setInfo(int i){ if( i < 3) ..... } %>
```

5. Scriptlet tag 지시어

Servlet6.0 및 JSP3.1 Tomcat10

개요

JSP 내에서 가장 많이 사용되는 중요한 태그이다.
서블릿의 doGet (doPost) 메서드내에서 처리했던 작업을 JSP에서도 사용 가능하게 지원하는 방법이며 변환될 때 _jspService 메서드내에 설정된다.
대표적인 작업으로 사용자 입력 파라미터 처리, 요청 포워드등과 같은 핵심 로직을 담당한다.

```
<%  
    자바코드 문장;  
    문장2;  
%>
```

샘플

```
<% if (Calendar.getInstance( ).get(Calendar.AM_PM) == Calendar.AM) {%}  
    Good Morning  
<% } else { %}  
    Good Afternoon  
<% } %>
```

6. expression tag 지시어

Servlet6.0 및 JSP3.1 Tomcat10

개요

선언 태그(declaration tag) 및 스크립트릿(scriptlet tag)에서 선언된 변수 및 메서드에서 리턴된 값을 웹 브라우저에 출력하는 방법이다.

<%= 변수 %>

<%= 리턴 값이 있는 메서드 %>

샘플

```
<%= (new java.util.Date( )).toLocaleString( ) %>
```

7. 내장 객체 (내장 변수)

Servlet6.0 및 JSP3.1 Tomcat10

개요

내장 객체는 JSP 페이지가 서블릿으로 변환될 때 항상 자동으로 추가되는 객체를 의미한다. 따라서 JSP 페이지에서 변수선언 없이 사용 가능하다.

내장 객체는 `_jspService` 메서드내에서 선언된 변수이기 때문에 `scriptlet tag` 및 `expression tag`에서만 사용이 가능하다.

내장 객체(내장 변수)	설 명
request	HttpServletRequest 객체 참조
response	HttpServletResponse 객체 참조
out	웹 브라우저 출력에 사용되는 JspWriter 객체 참조
session	HttpSession 객체 참조(session="true"인 경우에만 사용 가능)
application	ServletContext 객체 참조
config	ServletConfig 객체 참조
page	자바 클래스의 this와 동일
exception	발생되는 예외의 Throwable 객체에 대한 참조 (isErrorPage="true"인 경우에만 사용 가능)

변환된 java 파일

```
public void _jspService(final javax.servlet.http.HttpServletRequest request,
    final javax.servlet.http.HttpServletResponse response)
    throws java.io.IOException, javax.servlet.ServletException {

    final javax.servlet.jsp.PageContext pageContext;
    javax.servlet.http.HttpSession session = null;
    final javax.servlet.ServletContext application;
    final javax.servlet.ServletConfig config;
    javax.servlet.jsp.JspWriter out = null;
    final java.lang.Object page = this;
    javax.servlet.jsp.JspWriter _jspx_out = null;
    javax.servlet.jsp.PageContext _jspx_page_context = null;
```


8. EL (Expression Language)

Servlet6.0 및 JSP3.1 Tomcat10

개요

데이터를 웹 브라우저에 출력하기 위한 문법으로서 직관적이며 다른 출력방법과 비교하여 훨씬 사용하기 쉽게 구성되어 있다. (null 값 처리 용이)

일반적으로 서블릿에서 `setAttribute(key,value)` 메서드로 저장된 데이터 값을 브라우저에 출력하는 용도로 사용된다.

문법

`${ 표현식 }`

8. EL (Expression Language)

Servlet6.0 및 JSP3.1 Tomcat10

EL 연산자

연산자	설 명
.	자바빈 또는 Map에 접근할 때 사용
[]	배열 또는 List에 접근할 때 사용
()	우선순위 연산자
empty	값이 null인지 판단하는 연산자로서 true 리턴
+, -, *, /, %	산술 연산자 및 나머지 연산자
&&, , !	논리 연산자
=, >, >=, <, <=, !=	비교 연산자

EL 내장 객체

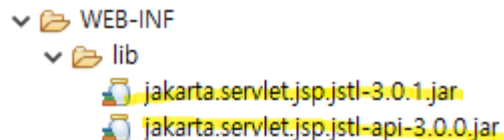
내장 객체	설 명
pageScope	page 영역에 존재하는 변수 참조 시 사용
requestScope	request 영역에 존재하는 변수 참조 시 사용
sessionScope	session 영역에 존재하는 변수 참조 시 사용
applicationScope	application 영역에 존재하는 변수 참조 시 사용
param	파라미터 값을 참조 시 사용
paramValues	파라미터 배열 값을 참조 시 사용
header	헤더 정보 값을 참조 시 사용
headerValues	헤더 배열 정보 값을 참조 시 사용
cookie	쿠키 정보를 참조 시 사용
initParam	context 초기화 파라미터 참조 시 사용
pageContext	pageContext 참조 시 사용

개요

JSP에서는 사용자가 필요하면 tag를 자체적으로 만들어 사용할 수 있다.
이러한 tag를 커스텀 태그(custom tag)라고 하며 Apache 그룹에서 매우 유용한 커스텀 태그를 제공하고 이것을 JSTL 이라고 부른다.
기존에 사용하던 javax 가 아닌 jakarta 패키지를 사용하는 JSTL 라이브러리를 사용해야 된다.

의존성 설정

```
<dependency>
  <groupId>jakarta.servlet.jsp.jstl</groupId>
  <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
  <version>3.0.0</version>
</dependency>
<dependency>
  <groupId>org.glassfish.web</groupId>
  <artifactId>jakarta.servlet.jsp.jstl</artifactId>
  <version>3.0.1</version>
</dependency>
```



제공 라이브러리

라이브러리	<u>uri</u>	prefix	예
core	<u>jakarta.tags.core</u>	c	< <u>c:tag</u> ..>
XML Processing	jakarta.tags.xml	x	< <u>x:tag</u> ..>
I18N formatting	<u>jakarta.tags.fmt</u>	<u>fmt</u>	< <u>fmt:tag</u> ..>
SQL	<u>jakarta.tags.sql</u>	<u>sql</u>	< <u>sql:tag</u> ..>
Functions	<u>jakarta.tags.functions</u>	<u>fn</u>	<u>fn:functions(...)</u>

```

<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<%@ taglib prefix="fn" uri="jakarta.tags.functions" %>
<%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>
<%@ taglib prefix="sql" uri="jakarta.tags.sql" %>
<%@ taglib prefix="x" uri="jakarta.tags.xml" %>

```

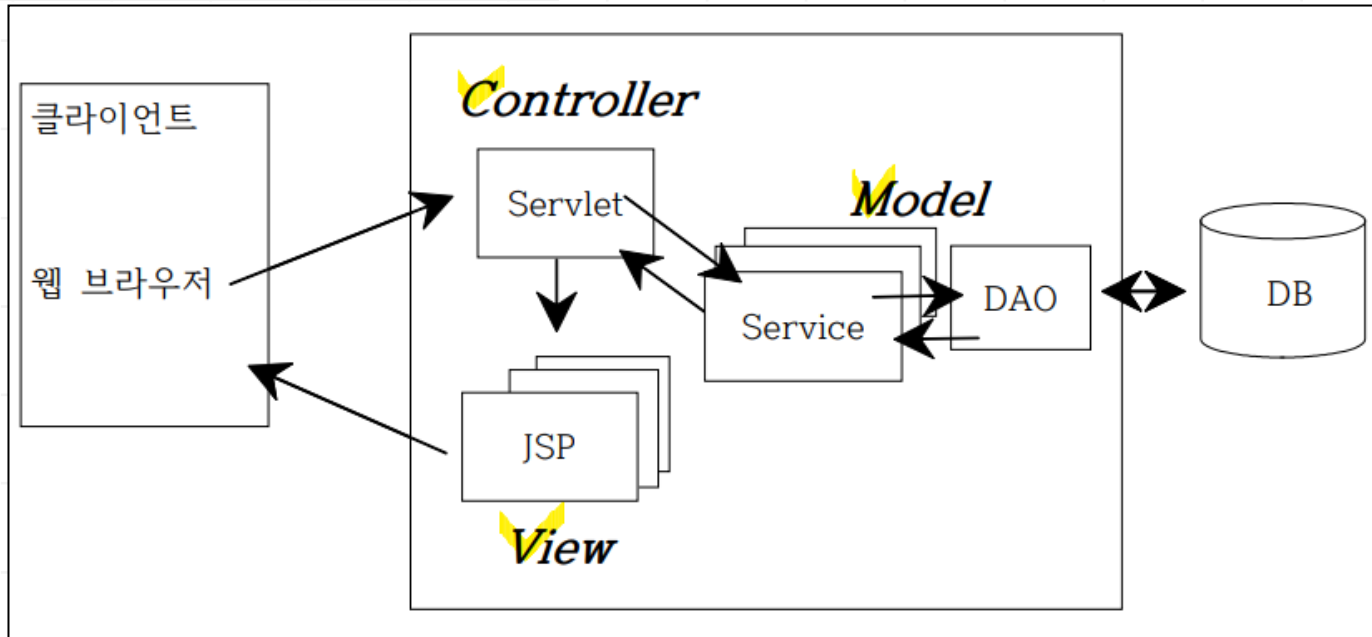
JSTL Core 라이브러리

태그	설명	사용예
out	지정된 값을 출력할 때 사용.	<c:out ... >
set	JSP의 setAttribute(name,key)기능과 동일. scope에 따른 데이터 저장/조회 가능하다.	<c:set ... >
remove	JSP의 removeAttribute(name)기능과 동일. scope에 따른 속성 제거가 가능하다.	<c:remove .. >
if	조건 처리를 사용할 때 사용.	<c:if .. >
forEach	반복 처리를 하고자 할 때 사용.	<c:forEach ..>
choose	자바의 switch문과 비슷.	<c:choose ..>
when	choose의 서브 태그로 사용. 조건을 만족한 경우에 사용된다.	<c:when>
otherwise	choose의 서브 태그로 사용. 조건을 만족하지 못한 경우에 사용된다.	<c:otherwise>
url	URL을 생성하는 기능.	<c:url>
forTokens	자바의 StringTokenizer 클래스 기능.	<c:forTokens>

개요

웹 어플리케이션을 개발할 때 비즈니스 로직과 프리젠테이션 로직을 명확하게 분리하여 설계하는 방법으로서 현업에서 사용되고 있는 패턴이다. (Model 2 Architecture)

MVC 아키텍처





감사합니다.
